江南大学 SHARK 战队

RoboMaster 2026 赛季

无线电雷达开源报告

从空中广播到裁判系统:

一套 2-GFSK 接收链路的完整实现与踩坑记录

郭兰鑫

江南大学霞客湾校区 · SHARK 战队

2026 年 8 月 13 日

摘要

本文记录 SHARK 战队 RoboMaster 2026 赛季无线电雷达的完整实现。本赛季规则在雷达基座旁引入官方发射源，向雷达广播对方队伍的位置、血量、发弹量与增益信息，同时在邻近频段发射一路强 50 dB 的干扰波。这条无遮挡的全场信息通路把雷达从“纯视觉识别”变成了“射频接收 + 视觉观测”的双源系统，也把射频前端、解调实现和链路诊断推到了工程量的第一位。

无线电是本赛季的全新方向，我们在几支队伍已公开的方案基础上继续摸索，本文记录这个过程，包括走错又走回来的部分。内容包括：2-GFSK 接收链路的逐级实现与每级参数的来源（含一处容易搞错的抽取灵敏度换算）；针对官方“强干扰紧贴弱信息波”频率规划的三层应对，以及为何自动增益控制在这个场景下必须关闭——这是我们绕弯路最多的一处；解调参数的自动调谐与粘滞锁定；利用干扰波恒定重发特性的时间分集接收，在单帧无法通过 CRC 时仍能恢复密钥；视觉与无线电的双源融合与比赛决策；以及一个把解析过程按阶段拆开计数的 Web 诊断面板，用于现场几十秒内定位故障环节。比赛全过程录制原始 IQ 与话题事件流，赛后可离线重放复现任意一帧的解调结果——文中多个参数的取值都是这样定下来的。

实测在实验室按规则手册标称功率复现发射时，三级干扰波下仍可稳定解析信息波，丢包率约 25%；赛场实际环境下可稳定以 10 fps 解析，丢包率在 10% 以下。

视觉雷达的检测、投影与轨迹预测部分大量参考了厦门理工 PFA 战队与香港科技大学 ENTERPRIZE 战队的开源工作，本文不再重复，只说明视觉与无线电之间的 ROS 2 通信接口与遥测契约。

关键词 无线电雷达；2-GFSK；软件无线电；GNU Radio；ROS 2；双源融合；双倍易伤

目录

目录 ii

图目录 iii

1 简介综述 1

1.1 现有开源工作与本文的位置 1

1.2 本文写了什么 1

1.3 系统概览 2

2 实现方法 3

2.1 空中协议 3

2.1.1 两路波与它们承载的东西 4

2.1.2 波形参数 4

2.1.3 空中帧结构 5

2.1.4 裁判帧如何切进空中包 6

2.1.5 六条命令的字段 7

2.1.6 裁判系统侧的相关命令 8

2.2 射频前端 9

2.2.1 天线选型 10

2.2.2 SDR 选型：不一定要 ANTSDR，但至少要带网口 10

2.2.3 频率规划与邻道干扰 11

2.2.4 关键射频参数 12

2.2.5 信息波为什么不能开 AGC 13

2.2.6 信息波用带通而不是低通 13

2.3 解调链路 14

2.3.1 链路总览 15

2.3.2 抽取与调频灵敏度：一处必须算清的换算 16

2.3.3 各级参数的选取 17

2.3.4 帧同步：容错与反相 18

2.3.5 自同步组帧器 18

2.3.6 丢包率如何估计 20

2.3.7 自动调谐与粘滞锁定 20

2.3.8 干扰波的时间分集 22

2.4 系统架构与通信 23

2.4.1 为什么用 ROS 2 包 GNU Radio 23

2.4.2 节点划分 25

2.4.3 视觉与无线电的通信契约 25

2.4.4 串口独占与协议限速 . . . 27
2.4.5 两个应用的边界 . . . 28
2.5 双源融合 . . . 28
2.5.1 时效模型 . . . 29
2.5.2 来源选择规则 . . . 29
2.5.3 阵营热同步与一个必须做的清理 . . . 30
2.5.4 0x0305 载荷的构造 . . . 31
2.6 比赛决策 . . . 31
2.6.1 双倍易伤自动决策 . . . 31
2.6.2 无敌目标广播 . . . 33
2.6.3 干扰密码生成 . . . 34
2.7 现场诊断：比赛接收台 Dashboard . . . 35
2.7.1 几个设计取舍 . . . 35
2.7.2 按阶段拆开的解析诊断 . . . 36
2.7.3 按现象定位：现场用法 . . . 37
2.7.4 架天线的正确用法 . . . 38
2.7.5 HTTP 接口 . . . 38
2.8 比赛录制与离线复现 . . . 39
2.8.1 为什么要录 IQ . . . 39
2.8.2 录制触发 . . . 39
2.8.3 存储策略 . . . 40
2.8.4 IQ 落盘格式 . . . 40
2.8.5 三条离线复现路径 . . . 41
3 硬件要求 41
3.1 本队配置 . . . 41
3.2 最低要求与替代方案 . . . 42
3.3 软件环境 . . . 42
3.4 部署要点 . . . 43
4 性能评估与测试 43
4.1 实测结果 . . . 43
4.1.1 实验室：按规则手册标称功率复现发射 . . . 43
4.1.2 赛场实际环境 . . . 44
4.2 测试策略 . . . 44
4.2.1 不需要硬件的部分（pytest） . . . 44
4.2.2 需要硬件的部分（实测脚本） . . . 45
4.2.3 虚拟链路自环 . . . 46

5 总结与展望 **46**
5.1 项目总结 46
5.2 已知不足 47
5.3 未来展望 48
6 鸣谢 **49**
7 后记 **49**
参考文献 **50**

图目录

1 比赛接收台 Dashboard 的雷达地图页 3
2 2-GFSK 接收链路的逐级结构 15
3 无线电监控页 36

1 简介综述

雷达这个兵种的技术路线一直跟着规则走。前几个赛季它基本等价于”一台俯视全场的相机 + 一套目标检测”，各队的开源也集中在检测网络、投影标定和轨迹跟踪上。2026 赛季规则手册第 5.6 节加了一条新机制：比赛过程中，双方雷达基座旁放置官方发射源，发射两路电磁波——一路**信息波**载有对方队伍的位置、血量、发弹量、增益和经济信息，一路**干扰波**载有一个六位密钥，用于难度对抗。红方基座的发射源播的是蓝方信息，蓝方基座播的是红方信息。

这条通路的性质和相机完全不同：它没有遮挡，不受光照影响，也不依赖标定精度，拿到的直接就是场地坐标（单位 cm）。代价是速率低（各命令 10 Hz）、内容固定（只有对方六台车），且完全受制于射频链路的好坏——天线没对准、增益不合适、邻道干扰压过来，都会让这条路直接归零。

于是雷达从”一个感知问题”变成了”两个感知源 + 一层融合”。

1.1 现有开源工作与本文的位置

视觉一侧的开源已经比较成熟。厦门理工 PFA 战队的单目方案 [6] 给出了多层透视变换的完整实现；香港科技大学 ENTERPRIZE 战队 [7] 在此基础上换成基于三维网格的射线投射定位，并自研了一套面向固定兵种数量的匈牙利匹配追踪器。我们的视觉链路（`vision/`）在检测、投影与轨迹预测上大量参考了这两份工作，因此本文不再重复展开，只说明它与无线电之间的通信接口。

无线电一侧是本赛季的新方向，目前公开的资料还比较零散。华南师范大学 PIONEER 战队 [3] 给出了整体方案的划分；北师港浸大 Navigator 战队 [4, 5] 的两篇分享集中在射频前端与硬件踩坑。这些资料帮我们省掉了大量试错时间，但也留下了几处空白：从 IQ 到裁判帧的每一级参数究竟该怎么定、邻道那个强 50 dB 的干扰波在软件层面如何应对、以及现场出问题时如何在几十秒内判断坏在哪一环。

本文想补的正是这几处。

1.2 本文写了什么

我们不认为自己的方案是最优解——赛季初期基本是在”能不能收到”这个层面挣扎，很多决定是试出来的而不是设计出来的。本文想做的是把这个摸索过程记录下来，特别是那些走错了又走回来的部分。具体来说有五部分内容：

1. **2-GFSK 接收链路的逐级实现，以及每级参数的来源。**从 gr-iio 取样、数字 AGC、二倍抽取、FM 鉴频、高斯滤波、Mueller & Müller 定时恢复到硬

判决，再到帧同步、CRC8/CRC16 自同步组帧与六条 `0x0A0x` 命令解析。用的都是常规手段，但我们尽量避免留下” 不知道为什么是这个值” 的常数——比如为什么用 2 MS/s 而不是规则给的 1 MS/s，以及做完二倍抽取后调频灵敏度为什么必须乘回 2。这一处当初卡了我们一阵，症状还很有欺骗性。

2. **对官方频率规划的应对**。规则把 −10 dBm 的干扰波放在距 −60 dBm 信息波仅 400 kHz 处（三级难度时），这是本赛季射频侧最主要的矛盾。我们的做法是空间选择性（定向八木）、幅度稳定性（固定增益而非 AGC）与频率选择性（实系数带通挖掉 DC）三层叠加。其中” 信息波必须关掉 AGC” 是我们绕了最多弯路才定位到的问题，本文完整记录了它的现象、当时的误判方向和真正原因。
3. **解调参数的自动调谐与粘滞锁定**。本振 ppm 偏差与发射源频率误差叠加后，实际载波会偏离标称值，换个场地就得重调。我们让接收端自己在（频偏，低通截止，SPS）三维网格上搜索能解出预期命令的参数组合，命中后锁住获胜组合并只做快扫，连续多次未命中才回退全量扫描。
4. **干扰波的时间分集接收**。干扰波以 10 Hz 重复发送同一份密码帧，这个特性可以利用。我们做了两级：软符号等增益合并（在比特判决之前平均，取得硬判决表决所不具备的约 2 dB 合并增益）与聚簇多数表决。软合并是通信里的常规做法，我们做的只是把它接到这个具体场景上，并守住”CRC 始终是唯一裁判” 这条底线——分集只增加候选，不放宽判据。
5. **双源融合、比赛决策与现场可诊断性**。按时效性分级决定每个目标的最终坐标来源，任一源失效仍能继续标记；在无线电侧统一做双倍易伤自动申请、无敌目标广播与干扰密码生成。配套一个把解析过程按阶段拆开计数的 Web 诊断面板（帧同步失败 / 长度域失败 / CRC16 失败 / 有效），使” 没收到信号” 和” 收到但解不出” 能被直接区分开。这个想法很简单，但对现场排查的帮助超出我们预期。

1.3 系统概览

整套系统分两个子系统，以 Git 子模块形式组织在同一个聚合仓库下。视觉一侧用一台海康工业相机俯视全场，输出目标在赛场坐标系下的位置；无线电一侧用两台 SDR 分别接收信息波与干扰波，解出对方全场信息；两者在无线电侧融合后，由单一进程统一向裁判系统发送指令。

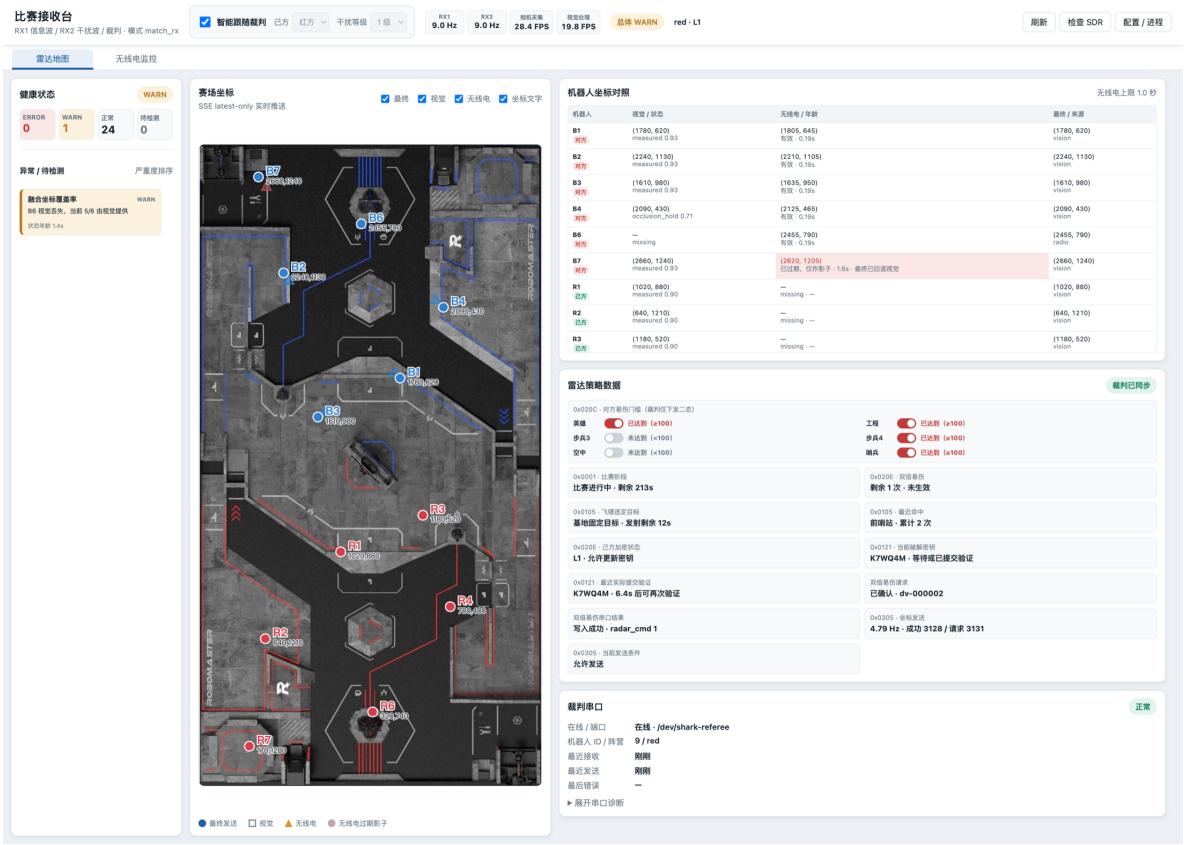


图 1: 比赛接收台 Dashboard 的雷达地图页，也是本项目最直观的一部分。左侧为健康状态与异常列表，中间为赛场地图上的三源叠加点位，右侧逐台列出每个目标的视觉坐标、无线电坐标与数据年龄、最终坐标与来源。图中 B6 是视觉丢失后由无线电接管的例子，B7 是无线电已过期、回退视觉的“过期影子”。本图及后文所有界面截图均为效果展示，内容由构造的模拟数据驱动，面板结构与协议字段与代码一致，但坐标、血量、帧率、密钥等数值都是编的，请勿当作实测性能或赛场数据引用。

本文余下部分的组织：第 2 节是实现方法的主体，按空中协议、射频前端、解调链路、系统架构与通信、双源融合、比赛决策、现场诊断、录制与复现的顺序展开；第 3 节给硬件要求；第 4 节是性能评估与测试策略；第 5 节总结、展望并列出已知不足。踩过的坑不单独设章，就写在对应的技术小节里。

2 实现方法

2.1 空中协议

这一小节把规则手册和通信协议里散落的参数集中整理一遍，并补上几处手册没直接写、但实现时必须搞清楚的细节。所有波形参数出自《RoboMaster 2026 机甲大师超级对抗赛比赛规则手册 V2.0.1》[1] 第 5.6 节与附录 1，命令字段出自《RoboMaster 2026 机甲大师高校系列赛通信协议 V2.0.0》[2] 第 1.6 节。

2.1.1 两路波与它们承载的东西

官方发射源同时发两路信号，用途完全不同。

表 1: 信息波与干扰波的对比

	信息波（广播源）	干扰波（干扰源）
承载内容	`0x0A01`–`0x0A05`，对方全部有效信息	`0x0A06`，六位 ASCII 密钥
推送速率	1400 byte/s，各命令 10 Hz	1350 byte/s，其中 `0x0A06` 为 10 Hz，其余为随机填充
功率强度	−60 dBm	−10 dBm
带宽	0.54 MHz	0.94 / 0.86 / 0.25 MHz（按难度等级）
对我方的价值	全场无遮挡敌情，融合的主要来源	解出密钥可提升对方干扰难度

这里有个反直觉的地方：**干扰波比信息波强 50 dB**。信息波是我们真正要的数据，功率却低得多；干扰波不含有用信息，反而是场上最强的那个信号，而且频点离信息波只有几百 kHz 到 1.7 MHz。这个功率与频率的组合决定了射频前端的全部设计（第 2.2 节），也是本项目在硬件上花时间最多的原因。

干扰波的对抗机制是这样运转的：开局双方干扰波默认一级难度。我方雷达解出对方干扰波里的六位密钥并上报赛事引擎，验证成功后，对方的干扰波难度提升一级——也就是我方雷达基座旁那个干扰源变强，我方接收信息波变难。反过来，若对方干扰难度高于我方，我方雷达即使准确标记也不会产生易伤，双倍易伤进度也不再累积。所以干扰密码不是可选功能，是必须做的。

2.1.2 波形参数

调制方式为 2-GFSK（二进制高斯频移键控），FSK 映射 $0 \to -1$，$1 \to +1$。八路波源（红蓝各一路信息波 + 各三级干扰波）的参数如下。

表 2: 八路波源的射频参数（规则手册表 5-23）

波源	中心频点/MHz	带宽/MHz	功率/dBm	Sensitivity
红方广播源	433.20	0.54	−60	1.5628
红方一级干扰源	432.20	0.94	−10	2.8194
红方二级干扰源	432.50	0.86	−10	2.5681
红方三级干扰源	432.80	0.25	−10	0.6517
蓝方广播源	433.92	0.54	−60	1.5628
蓝方一级干扰源	434.92	0.94	−10	2.8194
蓝方二级干扰源	434.62	0.86	−10	2.5681
蓝方三级干扰源	434.32	0.25	−10	0.6517

Sensitivity 单位为 rad/sample。八路共用 Sample Rate = 1 MS/s、SPS = 47、$BT = 0.35$。

其中 SPS（每符号采样数）= 采样率 / 符号率。2FSK 下每符号 1 bit，所以符号率 = 比特率 = 1,000,000/47 ≈ 21,277 bit/s。BT 是高斯滤波器的带宽-时间积。

Sensitivity（调频灵敏度）的定义是每个采样点的相位增量:

$$\text{Sensitivity} = \frac{2\pi\Delta f}{\text{Sample Rate}} \quad [\text{rad/sample}] \tag{1}$$

反解出信息波的频偏:

$$\Delta f = \frac{\text{Sensitivity} \times \text{Sample Rate}}{2\pi} = \frac{1.5628 \times 10^6}{2\pi} \approx 248{,}723 \text{ Hz} \tag{2}$$

这个数字和手册附录 1 给出的时域特征（bit= 1 瞬时频偏 +248,723 Hz，bit= 0 为 −248,723 Hz）一致，可以作为交叉验证。手册也给了另一个等价关系 $\Delta f = B/2 - \text{Sample Rate/SPS}$，代入信息波的 $B = 540$ kHz 得 270,000 − 21,277 = 248,723 Hz，同样吻合。

其余时域特征: 每 bit 持续 47 μs，高斯滤波器 3 dB 带宽 $BT/T_b = 0.35/47\ \mu\text{s} \approx$ 7,447 Hz。

Sensitivity 是与采样率绑定的量，不是波形的固有属性。式 (1) 的分母是采样率，所以同一个物理频偏在不同采样率下对应不同的 Sensitivity 数值。我们的接收机跑 2 MS/s，是标称值的两倍过采样，链路里做完 2 倍抽取后 Sensitivity 必须乘回 2 才对得上 1.5628。这一处踩过坑，第 2.3 节详细说。

2.1.3 空中帧结构

每个空中包定长 27 字节，结构极其简单:

表 3: 空中帧结构（规则手册附录 1）

字段	长度	值 / 说明
Access Code	8 字节（64 bit）	信息波 `0x2F6F4C74B914492E`，干扰波 `0x16E8D377151C712D`，大端序
Header	4 字节（32 bit）	两份相同的 Payload Length，各 16 bit 大端，值均为 15（`0x000F`）。接收端校验两份是否一致，不一致则丢包
Payload	15 字节	有效载荷

几点值得注意:

- **红蓝双方共用同一组 Access Code**。区分红蓝靠的是中心频点，不是接入码。接入码只用来区分信息波和干扰波两类。
- **没有独立前导码**，帧直接以 Access Code 起始。这意味着接收端没有一段已知的定长同步头可以做粗同步，只能直接在比特流里滑窗匹配 64 bit 接入码。
- **没有固定包间隔**，数据到达即发射。
- **没有数据白化，没有前向纠错**。这两条对实现影响很大——没有 FEC 意味着一旦有比特错就只能整帧丢弃，丢包率完全由信噪比决定；没有白化意味着长串相同比特会出现，定时恢复环路要能扛住这种情况。
- 高斯滤波器截断长度为 4 个符号周期（GNU Radio `digital_gfsk_mod` 的默认值）。

每包 $27 \times 8 = 216$ bit，持续 $216 \times 47\ \mu\text{s} = 10.15$ ms。

2.1.4 裁判帧如何切进空中包

这是实现时最容易出错的一处，手册写得很简短，但后果不小。

发射源要发的是标准的裁判系统串口帧（SOF、数据长度、seq、CRC8、cmd_id、data、CRC16），而空中包的 Payload 固定 15 字节。手册的规定是：**数据流按 15 字节为一组切片，每组作为一个空中包的 Payload**。

也就是说切片是在字节流层面做的，和裁判帧边界完全无关。一条 `0x0A01` 命令的帧长是 $5 + 2 + 24 + 2 = 33$ 字节，不是 15 的整数倍；信息波每 100 ms 推送 140 字节（1400 byte/s），140 也不是 15 的整数倍。两个都不整除的结果是:

> 一个裁判帧会跨越多个空中包，且跨越的相位每个周期都在变。

接收端因此**不能**假设”解出一个空中包就能拿到一条完整命令”。正确的做法是把所有解出的 15 字节 Payload 按顺序拼成一条连续字节流，再用一个

自同步的组帧器在这条流上找 SOF、验 CRC8、读长度、验 CRC16。我们的实现里RefereeFrameAssembler 就是这个角色（第 2.3.5 节）。

反过来，这个特性也让我们在实验室发射端必须刻意模拟它。flowgraphs/ganraoyuan/jiang.py 里维护的是一个字节队列而不是包队列：

```
def _next_broadcast_payload(self) -> List[int]:
    # V2.0.0 carries 140 bytes every 100 ms. Because 140 is not divisible
    # by the 15-byte air payload, retain the remaining 5/10 bytes and let
    # subsequent referee-frame cycles cross air-packet boundaries.
    while len(self._broadcast_byte_queue) < _AIR_PAYLOAD_SIZE:
        self._append_broadcast_period()
    return [self._broadcast_byte_queue.popleft()
            for _ in range(_AIR_PAYLOAD_SIZE)]
```

干扰波那一路还额外在队列开头塞了一段随机长度（0–14 字节）的填充，让裁判帧起始位置相对空中包边界随机偏移——因为官方的干扰波也是先推字节再切片，帧边界和包边界之间没有固定关系。如果发射端图省事让每个裁判帧都对齐到包边界，接收端就会在一个”过于友好”的信号上调通，到了赛场反而解不出来。

2.1.5 六条命令的字段

表 4: 雷达无线链路命令一览

cmd_id	名称	长度	内容
0x0A01	enemy_position	24	对方六台机器人的 x, y 坐标，各 2 字节，单位 cm
0x0A02	enemy_hp	12	对方英雄 / 工程 / 3 号 / 4 号步兵 / 保留 / 哨兵血量
0x0A03	enemy_bullet_allowance	10	对方英雄 / 3 号 / 4 号步兵 / 空中 / 哨兵允许发弹量
0x0A04	enemy_macro_status	8	剩余经济、总经济、增益点占领位图（32 bit）
0x0A05	enemy_buff_status	41	五台机器人的增益详情、哨兵模式、五台机器人主要状态
0x0A06	interference_password	6	六个 ASCII 字母或数字

解析实现在core/rm_protocol.py，字段范围约束（用于实验室发射端生成合法载荷）在core/radar_wireless_constraints.py。几条实现上的说明：

0x0A01 的坐标顺序是固定的六台车。 依次为对方英雄、工程、3 号步兵、4 号步兵、空中、哨兵。注意有空中机器人，这和 0x0A02 血量（第 5 个位置是保留字段）、0x0A03 发弹量（没有工程）的顺序都不一样。三张表不能混用同一个名字数组，我们在代码里分别定义了RADAR_WIRE_ROBOT_NAMES 和各命令自己的名字列表。

0x0A05 的 41 字节里最有用的是最后 5 个。 前 35 字节是五台机器人的增益详情（每台 7 字节：血量回复百分比、射击热量冷却、防御加成、负防御加成、攻击加成），第 36 字节是哨兵模式，最后 5 字节是五台机器人的主要状态：

表 5: 0x0A05 主要状态取值

值	枚举名	含义
0	alive	存活
1	destroyed	已被击毁
2	invincible_not_weakened	无敌且未被易伤
3	invincible_weakened	无敌且已被易伤

这 5 个字节驱动了两个功能：一是向本方机器人广播哪些目标当前无敌（第 2.6.2 节），二是双倍易伤决策时排除已阵亡和无敌的目标（第 2.6.1 节）——对一台已经无敌的车触发双倍易伤是纯粹浪费机会。

0x0A06 是六个 ASCII 字符，不是六字节二进制。 手册明确”每个字节均为 ASCII 码编码的字母或数字”。解析时我们做严格校验：decode('ascii', errors='strict') 加isalnum()，不满足就标记password_is_alnum_ascii = False。这一步在实践中很有用——如果解出来的”密钥”里有不可打印字符，那就是解调链路有问题而不是密钥本身特殊，能立刻排除方向。

2.1.6 裁判系统侧的相关命令

无线电只负责收，发指令还是走裁判串口。为了让后文的决策部分能对上，这里先列出我们用到的裁判系统命令：

表 6: 雷达用到的裁判系统串口命令

cmd_id	方向	用途
0x0001	收	比赛状态，用于判断比赛阶段与剩余时间，驱动录制起停
0x0003	收	全场机器人血量，用于本方英雄掉血检测
0x0201	收	本方机器人状态，其中robot_id 用于自动判断阵营
0x020C	收	雷达标记进度，逐机器人易伤门槛
0x020E	收	雷达自主决策同步，见下表
0x0305	发	地图机器人数据，即标记坐标，最高 5 Hz
0x0301	发	机器人交互数据。data_cmd_id = 0x0121 是雷达指令（消耗双倍易伤 / 提交密钥），0x0234 是我们自定义的无敌目标广播

0x020E 这一字节的位域拆解值得单独说，因为它同时驱动了三个功能：

表 7: 0x020E radar_info 位域

位	字段	用途
0–1	double_vulnerability_count	剩余双倍易伤机会（0–2），**双倍易伤决策的权威确认信号**
2	is_double_vulnerability	当前是否正在生效
3–4	opponent_interference_difficulty	对方干扰难度等级，驱动 RX2 频点与带宽热切换
5	can_change_password	是否允许更换密钥，上升沿触发生成新密码
6–7	保留	—

第 0–1 位是”剩余机会计数”而不是”是否成功”，这个细节在双倍易伤状态机里很关键——我们用计数下降来确认请求被受理，而不是用第 2 位的生效边沿。原因见第 2.6.1 节。

2.2 射频前端

无线电雷达的成绩上限在硬件选型和天线架设上，软件只能在这个上限之下发挥。这一小节是我们实测下来最值得照抄的部分，也是投入产出比最高的地方——把天线极化搞对带来的收益，比在解调环路上继续调参数大一个量级。

2.2.1 天线选型

信息波用定向八木。 理由很直接：官方发射源位置固定（就在雷达基座旁），信息波是这套系统最关键的数据来源，而它的功率只有 −60 dBm，比干扰波低 50 dB。这三个条件叠起来就是”值得用定向天线换增益”。

六单元八木除了增益，还有一个常被忽略的好处：**前后比**。它对来自其他方向的信号有天然抑制，而干扰波的发射源位置和信息波不完全重合，所以指向信息波的同时也在一定程度上压低了干扰波。这比在软件里加一级滤波便宜得多，也不引入群延迟。

架设有三个要点，按重要性排序：

1. **极化方向必须与发射源一致（垂直极化）**。这是最容易被忽略、代价又最高的一条。极化失配的理论损耗在正交时趋于无穷，实际由于交叉极化分量存在通常也在十几 dB 量级——比六单元和三单元八木之间的增益差距大得多。换句话说，**极化搞错的八木不如极化搞对的鞭状天线**。
2. **必须正对发射源**。八木的波束窄，指向偏了增益就白给。我们的做法是架好后用 Dashboard 的解析诊断面板边转边看（第 2.7.4 节），找到有效帧计数最高的方位再锁死，而不是靠目视对准。
3. **远离己方干扰源**。尽量避开自家电源、图传和另一台 SDR。这一条在实验室调试时尤其明显——四台 SDR 挤在一张桌子上时，接收端底噪会明显抬高。

干扰波用全向。 干扰波只需要解出六位密钥，数据量极小（`0x0A06` 只有 6 字节），而且它的功率是 −10 dBm，是场上最强的信号，对信噪比的要求远低于信息波。另一方面它的来源方向不像信息波那么确定。这两条加起来，全向天线是更务实的选择——省掉对准这一步，少一个出错环节。我们用的是玻璃钢全向天线，好处是结构结实、耐户外、架设方便。

2.2.2 SDR 选型：不一定要 ANTSDR，但至少要带网口

芯片层面，AD9361 的板子都兼容。 本项目的 flowgraph 通过 gr-iio 与 libad9361 访问设备，不依赖 ANTSDR 特有的任何东西。普通 ADALM-Pluto 及各类 Pluto 兼容板（含各种第三方改版）都能直接用，只需要改`config.yaml` 里的 URI。

但纯 USB 供电的板子不要用在比赛机上。 这一条要说得务实一点，因为它涉及硬件损坏：

- **USB 2.0 供电余量本来就紧**。AD9361 加 FPGA 满载取流时电流会顶到上限。

- **同一条线既供电又跑连续 I/Q**。我们的接收采样率是 2 MS/s 复数样本，遇到线材质量一般或主机 USB 控制器带宽紧张，就会出现丢样本、掉设备、`iio_readdev` 中断。这类故障的现象是" 解析成功率突然归零"，而且现场很难和天线问题区分开。
- **烧毁不是个例**。交流中多所学校反馈过纯 USB 供电的 Pluto 在长时间连续取流下损坏。我们不建议在比赛机上冒这个风险。
- **网口方案稳定得多**。走以太网（或 USB 网络 + 独立供电）把供电和数据分开，长时间连续取流的稳定性完全是另一个量级。

选型建议：**带千兆网口 + 独立供电的 AD9361 板子**。ANTSDR E310 是我们用的型号，其他满足这个条件的板子同样可以。

URI 一律用`ip:`形式，不用`usb:X.Y.Z`。后者随 USB 总线枚举顺序变化，重插一次设备就会变，配置文件跟着失效。`ip:` 地址是固定的，换机器部署时改一次就够。本项目所有配置里的 SDR URI 都是`ip:`形式，这也是选带网口板子的附带好处。

2.2.3 频率规划与邻道干扰

两路 RX 的中心频点随阵营切换，干扰波还随难度等级切换。全部定义在`core/radio_config.py`，Dashboard 的高级抽屉里只读显示当前生效值。

表 8: 两路 RX 的频率规划与接收带宽设置

用途	红方/MHz	蓝方/MHz	接收带宽设置
信息波（RX1）	433.200	433.920	540 kHz 收发带宽 / 260 kHz 带通
干扰波 · 一级	432.200	434.920	940 kHz / 500 kHz 低通
干扰波 · 二级	432.500	434.620	860 kHz / 500 kHz 低通
干扰波 · 三级	432.800	434.320	250 kHz / 160 kHz 低通

把频点画在一条轴上就能看清问题所在。以红方为例：

432.20 MHz	432.50 MHz	432.80 MHz	$\longrightarrow$	433.20 MHz
一级干扰	二级干扰	三级干扰		**信息波**
−10 dBm	−10 dBm	−10 dBm		**−60 dBm**

信息波与最近的三级干扰波只差 **400 kHz**，而干扰波比它强 **50 dB**。蓝方是镜像的情况（干扰波在信息波上方，最近的三级干扰在 434.32 MHz，同样差 400 kHz）。

这就解释了为什么难度等级越高越难解：一级干扰在 432.20 MHz，离信息波 1 MHz，带宽 940 kHz，虽然宽但离得远；三级干扰在 432.80 MHz，只差 400 kHz，虽然带宽收窄到 250 kHz，但主瓣边缘已经贴上来了。规则设计上”提升一级难度”的物理含义就是把强干扰往信息波推近。

对应的三层应对措施：

1. **空间选择性**——定向八木换增益与前后比（上一小节）；
2. **幅度稳定性**——固定增益而不是 AGC（下一小节）；
3. **频率选择性**——260 kHz 实系数带通（第 2.2.6 节）。

三层都是必要的，缺任何一层在三级干扰下都会明显退化。

2.2.4 关键射频参数

表 9: 接收端关键参数与依据

参数	值	依据
接收采样率	2 MS/s	空中波形 1 MS/s 的 2 倍过采样，见下
`input_sps`	94	空中 SPS = 47 对应 2 倍过采样
RX RF 端口	`B_BALANCED`	AD9361 的平衡输入
信息波增益模式	`manual`，32 dB	见第 2.2.5 节
信息波增益上限	73 dB	部署的 AD9361 报告 `hardwaregain_available=[-1 1 73]`
信息波滤波	260 kHz 带通	见第 2.2.6 节
干扰波增益模式	`fast_attack`	突发的常量密码帧，快速起攻更容易抓到

为什么用 2 倍过采样。 规则给的是 1 MS/s、SPS = 47。直接按 1 MS/s 接收当然可以，但 2 MS/s 有两个好处：

- **定时恢复的分辨率更高。** 47 个采样点一个符号，定时误差检测器能分辨的相位精度是 1/47 符号；94 个点就是 1/94。对没有 FEC 的链路，定时抖动直接转成比特错。
- **给频偏留余量。** AD9361 的本振有 ppm 级偏差，加上官方发射源自己的频率误差，实际载波可能偏离标称值。2 MS/s 的观测带宽让自动调谐（第 2.3.7 节）有空间在频率轴上搜索，而不会把信号搜到带外去。

代价是 CPU 和 IQ 数据量翻倍。数据量的问题在录制侧解决——落盘时整数抽取回 1 MS/s（第 2.8.4 节）。

2.2.5 信息波为什么不能开 AGC

这是本项目在射频侧踩过的最有代表性的一个坑，值得详细说。

最初信息波那一路用的是`slow_attack` 自动增益。现象是：解析成功率**忽高忽低**，几秒钟一个周期，天线不动也会变。一开始怀疑是天线在波束边缘或者发射源功率波动，转天线、换位置都没有稳定改善。

真正的原因是邻道那个强 50 dB 的干扰波。AD9361 的 AGC 看的是**整个接收带宽内的总功率**，而 2 MS/s 的观测带宽足够把干扰波一起收进来。干扰波本身是断续的数据流，总功率随之起伏，AGC 就跟着持续调整增益。增益一动，两件事跟着坏：

1. **定时恢复的环路增益变了。** M&M 定时误差检测器的输出幅度与信号幅度相关，增益跳变相当于突然改变环路增益，定时环需要重新收敛。
2. **自动调谐锁定的参数漂了。**我们的自动调谐会锁住一组获胜的（频偏，低通）参数（第 2.3.7 节）。增益一变，原本能解出帧的参数组合可能就不再最优，触发回退全量扫描，延迟又上去了。

改成`manual` 固定 32 dB 之后，成功率立刻变得平稳。这个值是台架上扫出来的——`apps/lab_tx/run_rx1_gain_sweep.sh` 就是干这个的，它遍历增益档位并统计各档下的有效帧计数。

只发命令不回读，遇到设备状态异常会静默失效。启动器不仅强制要求 manual 模式，还通过 AD9361 readback 验证模式和增益值都真的写进去了。这一层校验加上之后确实抓到过设置没生效的情况——如果只发`set_gain_mode()` 就认为成功，现场会以为是天线问题，浪费掉宝贵的排查时间。

干扰波那一路相反，用`fast_attack`。它只要解出六位密钥，每 100 ms 重复发一次同样的内容（第 2.3.8 节会利用这个特性），而且信号强，AGC 抖动带来的问题远小于”快速起攻抓住突发”的收益。

2.2.6 信息波用带通而不是低通

RX1 和 RX2 的滤波结构不一样，这是一处有意的区别。

RX2（干扰波）走的是两级级联低通，500 kHz 或 160 kHz 截止——这是从早期版本延续下来的、已经验证可用的路径。

RX1（信息波）用的是**实系数带通**，通带 5 kHz–260 kHz，过渡带 50 kHz：

```
BROADCAST_BPF = {
    "red":  {"low_cut": 5_000, "high_cut": 260_000, "transition": 50_000},
    "blue": {"low_cut": 5_000, "high_cut": 260_000, "transition": 50_000},
}
```

用带通的理由是要**挖掉 DC 附近那一块**。AD9361 零中频接收机在直流附近有本振泄漏和 DC 偏置残留，表现为频谱中心一根尖峰。信息波是 GFSK，按式 (2)，± 248.7 kHz 处才是主要能量所在，DC 附近这根尖峰对解调没有贡献，只会抬高鉴频器输出的直流分量、干扰后面 AGC 的参考电平估计。低通留着它，带通去掉它。

用**实系数**（`fir_filter_ccf`，实数抽头作用于复数样本）而不是复系数的原因是：实抽头滤波器对正负频率的响应是对称的，所以一个 5–260 kHz 的实系数带通实际同时保留了 $[-260, -5] \cup [+5, +260]$ kHz 两侧——正好是 GFSK 的两个主瓣，同时排除中间的 DC 区。这比设计一个复系数带通简单，也比先低通再单独去 DC 少一级滤波。

260 kHz 这个上限来自 $\Delta f = 248.7$ kHz。理论上要装下 GFSK 的完整频谱还需要更宽，但 $BT = 0.35$ 的高斯滤波已经把旁瓣压得很低，260 kHz 能装下主瓣的绝大部分能量，同时把 400 kHz 外的三级干扰波挡在过渡带之外。这是一个”够用就好”的取舍——再放宽会引入更多干扰，收窄会切掉有用能量。

2.3 解调链路

从 IQ 采样到一条可用的裁判命令，中间要过五道关：解调出比特、在比特流里找到空中包、把包拼成字节流、在字节流里组出裁判帧、校验通过。每一道都可能失败，而且失败原因完全不同——这也是为什么诊断要按阶段拆开（第 2.7.2 节）。

2.3.1 链路总览

级	作用
gr-iio 取样（2 MS/s 复数）	从 AD9361 连续取流
↓ 实系数带通 5–260 kHz	抗邻道 + 去 DC（第 2.2.6 节）
↓ 复数 AGC	归一化幅度，为鉴频器提供稳定输入
↓ 二倍抽取	2 MS/s → 1 MS/s，SPS 94 → 47
↓ 求积鉴频器	相位差分，FM → 基带电平
↓ 高斯滤波器 $BT = 0.35$	与发射端匹配，抑制符号间干扰
↓ 浮点 AGC	归一化到判决门限附近
↓ M&M 定时恢复	符号同步，输出每符号一个采样
↓ 二值硬判决	电平 → 比特
接入码滑窗匹配	找 64 bit Access Code，容 Hamming 距离
↓ 长度域校验	验 `0x000F000F`，两份是否一致
↓ 取 15 字节 Payload	拼进连续字节流
自同步组帧器	找 SOF、验 CRC8、读长度、验 CRC16
↓ 命令长度白名单	长度与 cmd_id 是否匹配
↓ 字段解析	按第 2.1 节的表拆字段

图 2: 2-GFSK 接收链路的逐级结构。上半部分（到硬判决）在 GNU Radio flowgraph 里，实现于`flowgraphs/gfsk/continuous_gfsk_demod.py`；下半部分是纯 Python，实现于`core/rm_protocol.py`，不依赖 GNU Radio，因此可以脱离硬件单独测试。

我们同时保留了两条解调路径：

- **信息波用`mm`** ——上面这条连续流路径，高斯滤波 + Mueller & Müller 定时恢复。这是本赛季新写的，也是比赛实际使用的。
- **干扰波用`legacy`** ——GNU Radio 自带的`digital.gfsk_demod`。它内部是 Clock Recovery MM 的旧实现，参数不如新路径可控，但在干扰波这种高信噪比场景下已经足够，而且经过了更长时间的验证，没有必要为了统一而改。

另外还有一条 **IQ 快照解码器**作为可显式打开的诊断回退路径：它把一段 IQ 缓存下来，用纯 NumPy 重新走一遍解调（实现在`core/offline_iq_scan.py`）。比赛配置里默认关闭（`iq_snapshot_enabled: false`），因为它会产生重复帧、而且是第二个吃 CPU 的解码器。但它在两个场合非常有用：一是自动调谐要在参数网格上反复试解（第 2.3.7 节），二是赛后离线复现（第 2.8.5 节）。同一份解码逻辑服务三个用途，这也是把它写成纯 NumPy 而不是 GNU Radio 块的原因。

2.3.2 抽取与调频灵敏度：一处必须算清的换算

链路里做二倍抽取有个明确的动机：让定时恢复环路工作在规则手册的标称时基上。

设备跑 2 MS/s、SPS = 94，抽取 2 之后变成 1 MS/s、SPS = 47，正好是手册给的参数。这样做的好处是鉴频器的灵敏度可以直接对上手册的 1.5628，不需要在环路参数里带一个” 我们是两倍采样” 的隐含约定。

但换算必须做对。求积鉴频器的输出正比于相邻采样点之间的相位差：

$$y[n] = \arg\left(x[n] \cdot x^*[n-1]\right) \tag{3}$$

抽取 D 倍之后，相邻输出样本之间跨越了原始信号的 D 个采样周期，所以每个输出样本承载的相位增量是原来的 D 倍：

$$\text{Sensitivity}_{\text{out}} = D \times \text{Sensitivity}_{\text{in}} \tag{4}$$

代码里的处理是：构造时传入`input_sensitivity` $= 1.5628/2$，即”2 MS/s 下的灵敏度”；抽取后的输出灵敏度按式 (4) 乘回 2，恰好等于 1.5628；鉴频器增益取其倒数：

```
@property
def output_sensitivity(self) -> float:
    # Phase change per output sample grows by the decimation factor.
    return self._input_sensitivity * float(self._decimation)

def _discriminator_gain(self) -> float:
    return 1.0 / self.output_sensitivity
```

这里搞错的症状很有欺骗性。如果灵敏度算错 2 倍，鉴频器输出的幅度就整体缩放了 2 倍。由于后面紧跟一级浮点 AGC，幅度会被重新归一化，所以**频谱看起来正常、AGC 也不报错**，帧同步甚至偶尔能过——因为接入码匹配允许 Hamming 距离容差。最终表现是” 能收到一些帧但 CRC16 大量失败”，很容易被误判成信噪比不够，于是往调增益、转天线的方向去查。真正的判据是看 CRC16 失败率是否随信噪比改善而下降——如果无论怎么调都停在一个高位，就该回头查这类换算问题。

2.3.3 各级参数的选取

表 10: 连续解调链路的参数

参数	值	说明
`input_sps`	94	2 MS/s 下的每符号采样数
`decimation`	2	抽取到手册标称的 1 MS/s / SPS 47
`input_sensitivity`	1.5628/2	见第 2.3.2 节
`bt`	0.35	与发射端匹配（手册规定）
`gaussian_taps`	60	约 1.3 个符号周期长度
`loop_bandwidth`	0.046	M&M 环路带宽（归一化）
`damping_factor`	1.0	临界阻尼
`ted_gain`	0.85	定时误差检测器增益
`max_deviation`	1.5	允许的最大定时偏移（符号）
复数 AGC 速率 / 上限	10^{-4} / 4.0	慢速跟踪，避免跟着干扰波抖
浮点 AGC 速率 / 上限	10^{-4} / 65536	判决前归一化

几处值得说明：

两级 AGC 的速率都设得很慢（10^{-4}）。 理由和第 2.2.5 节关掉硬件 AGC 是同一个——邻道干扰波的功率在起伏，AGC 跟得越快，判决门限抖得越厉害。这两级 AGC 的作用是补偿慢变的链路增益（天线指向微调、温漂），不是跟踪瞬时包络。

复数 AGC 的最大增益限制在 4.0。 这一级放在带通之后、鉴频之前。如果不限幅，在没有信号的间隙 AGC 会一路把增益推到上限放大底噪，等信号回来时又要花时间收回去。限制在 4 倍相当于给它一个合理的动态范围窗口。

M&M 用 BPSK 星座而不是自定义的 $[-1,+1]$ 星座。 `digital.symbol_sync_ff` 需要一个星座对象来做判决。参考实现里用的是`calcdist` 构造的两点星座，我们改用`digital.constellation_bpsk()` ——两者在数学上等价（都是两点实数

判决），但后者在更多 GNU Radio Python 绑定版本里可用，减少一个跨版本兼容问题。

高斯滤波抽头数 60。 $BT = 0.35$、SPS $= 47$ 时，高斯脉冲的有效长度约 1.3 个符号周期。60 个抽头（≈ 1.28 符号）刚好覆盖主要能量。取更长会增加群延迟和计算量，收益很小。

2.3.4 帧同步：容错与反相

比特流出来之后要找 64 bit 的 Access Code。由于没有前导码，这一步只能滑窗匹配。实现在`AccessCodeAirPacketExtractor`，有两处刻意的容错：

允许 Hamming 距离容差（默认 3）。 64 bit 里容 3 bit 错。这个数字是权衡的结果：放太严会在信噪比边缘丢掉本来能救回来的帧（因为后面还有 CRC 兜底），放太松会产生大量虚假同步——每个虚假同步都要走一遍长度域校验和 CRC，浪费 CPU。64 bit 随机序列落在某个码字 Hamming 距离 ≤ 3 内的概率是 $\sum_{k=0}^{3}\binom{64}{k}/2^{64} \approx 2.4 \times 10^{-15}$，虚警可以忽略。

允许反相变体。 FM 鉴频器的输出极性取决于本振在信号的哪一侧（频偏符号），而这一点在有频偏的情况下可能翻转——表现为所有比特整体取反。提取器为每个接入码同时注册正相和反相两个模式，匹配到反相变体时把后续的长度域和载荷比特也一并取反：

```
for pattern in self.access_patterns:
    self.access_variants.append((pattern, pattern, False))
    if bool(allow_inverted):
        self.access_variants.append(
            ([1 - bit for bit in pattern], pattern, True))
```

这一条在实践中救回过整条链路。刚开始调的时候遇到过”频谱明明有信号、接入码怎么都匹配不上”的情况，加上反相变体后立刻通了。如果没有这个机制，就得靠人去猜频偏方向然后手动改符号，现场根本来不及。

长度域校验是严格的（容差 0）。 接入码之后的 4 字节必须严格等于 `0x000F000F`。这里不给容差，是因为它是一道低成本的二次确认：接入码放宽了容差，虚假同步虽然概率低但不是零，长度域严格匹配可以在花 CRC 计算之前把它们筛掉。手册也明确要求”两份不一致则丢包”。

2.3.5 自同步组帧器

如第 2.1.4 节所述，裁判帧跨空中包边界，所以组帧必须在字节流上做。`RefereeFrameAssembler` 的循环逻辑是：

算法 1 自同步裁判帧组帧

while 缓冲区非空 **do**
  找到第一个 `0xA5`（SOF）；找不到则清空缓冲返回
  丢弃 SOF 之前的字节
  if 剩余不足 5 字节 **then**
    返回，等更多数据
  end if
  取前 5 字节为帧头
  if CRC8(header[0:4]) $\neq$ header[4] **then**
    记录`crc8_failure`；丢弃 1 字节；**continue**
  end if
  $L \leftarrow$ 帧头里的小端 16 位数据长度
  $N \leftarrow 5 + 2 + L + 2$
  if $N < 9$ **then**
    记录`length_failure`；丢弃 1 字节；**continue**
  end if
  if 剩余不足 N 字节 **then**
    返回，等更多数据
  end if
  if CRC16(raw[0:$N-2$]) $\neq$ 末尾两字节 **then**
    记录`crc16_failure`；丢弃 1 字节；**continue**
  end if
  取出 cmd_id 与 data，从缓冲区移除整帧
  if cmd_id 的期望长度 $\neq L$ **then**
    记录`command_length_failure`；**continue**
  end if
  记录`valid`；产出一帧
end while

三处设计值得说：

校验失败时只丢 1 字节，不丢整帧。 因为”看起来像 SOF 的字节”可能本来就是载荷里的 `0xA5`。只前进一个字节，让真正的帧起点有机会被找到。代价是最坏情况下要多做几次 CRC，但这个开销远小于丢掉一整帧真实数据。

命令长度白名单。 CRC16 通过之后还要检查`cmd_id` 对应的数据长度是否符合预期（第 2.1 节那张表）。这一层看起来冗余，但它能挡住一类特殊情况：随机填充字节偶然构成一个 CRC 合法的帧。CRC16 的碰撞概率是 2^{-16}，干扰波每秒 1350 字节的随机填充意味着每秒有上千个 SOF 候选，长期跑下来一定会撞上。白名单把这类”合法但不可能存在”的帧挡在外面。

诊断事件在这里产生。 每次判定都记一条事件（`crc8_failure` / `length_failure` / `crc16_failure` / `command_length_failure` / `valid`），由`ReceptionMetrics` 做滑动窗口统计。这是整个诊断体系的数据源——因为只有在这个位置才知道一帧是在哪一步死的。

2.3.6 丢包率如何估计

裁判帧头里有一个 8 位序号（seq），逐帧递增。利用它可以估计**在解调之前就丢掉**的帧数——也就是那些连 CRC 判定都没走到的帧。

做法是：只统计走到了 CRC16 判定这一步的帧（无论通过与否），因为只有它们的 seq 是可信的（受 CRC8 保护）。相邻两帧的 seq 差 $\delta = (s_i - s_{i-1}) \bmod 256$：

- $\delta = 0 \rightarrow$ 重复帧
- $1 \le \delta \le 127 \rightarrow$ 正常前进，中间丢了 $\delta - 1$ 帧
- $\delta > 127 \rightarrow$ 判为乱序，不参与统计

于是：

$$\text{丢包率}_{\text{解调前}} = \frac{\sum(\delta_i - 1)}{\sum \delta_i} \tag{5}$$

再加上通过了 CRC8 但被 CRC16 或长度白名单拒掉的帧，得到端到端丢包率：

$$\text{丢包率}_{\text{端到端}} = \frac{\sum(\delta_i - 1) + N_{\text{rejected}}}{\sum \delta_i} \tag{6}$$

第 4 节引用的丢包率数字就是这么算的。

这个估计有两处固有偏差，引用时要知道。第一，seq 是 8 位的，连续丢超过 127 帧会被判成乱序而漏掉——但在 10 Hz 的推送速率下这相当于连续断 12 秒以上，那种程度的中断有别的指标能发现。第二，链路空闲一段时间后重新有信号时，seq 会出现一个大跳变，这不是丢包而是新会话。我们用一个空闲阈值（默认 2 秒）来切分会话，超过阈值就重新起算，不把跳变计入丢失。

2.3.7 自动调谐与粘滞锁定

AD9361 的本振有 ppm 级偏差，官方发射源自己也有频率误差，两者叠加后实际载波会偏离标称频点。SPS 同理——收发两端的采样时钟不同源，实际的每符号采样数会略微偏离 94。

如果这些偏差固定，手工调一次就够。问题是它们会随温度和设备变化，换个场地、换台机器就得重调。所以我们让接收端自己搜。

搜索空间是三维的：

$$(\text{频偏}\Delta_f,\ \text{低通截止}f_c,\ \text{SPS}) \tag{7}$$

- **频偏候选**：请求值、0、以及从 IQ 的 FFT 峰值估计出的偏移（各自再 ±25 kHz）。信息波在高难度等级下还额外加入 −350 到 +300 kHz 的一组固定候选。
- **低通候选**：基准值、220 / 260 / 320 / 380 kHz，按难度等级选不同子集。三级干扰下收窄到 220 kHz 有时反而更好，因为干扰贴得近。
- **SPS 候选**：围绕标称值以 0.5 为步长展开，即`92`，`92.5`，`93`，`93.5`，`94`，`94.5`，`95`。

每个候选组合都在一段 IQ 上试解，然后按下面的评分排序：

$$\begin{aligned}\text{score} = {} & 10^7 \cdot \mathbb{1}[\text{期望命令全部解出}] \\ & + 10^6 \cdot N_{\text{期望命令种类}} \\ & + 10^4 \cdot N_{\text{裁判帧}} \\ & + 10^3 \cdot \mathbb{1}[\text{接入码类型匹配}] \\ & + N_{\text{空中载荷}} - d_{\text{Hamming}}\end{aligned} \tag{8}$$

权重的数量级差距是刻意的，表达一个严格的优先顺序：**能不能解出全部期望命令** ≫ 解出几种 ≫ 解出几帧 ≫ 接入码对不对 ≫ 细节。这样即使某个参数组合解出了一大堆帧，只要它没覆盖全部期望命令，也排不到只解出少量帧但命令完整的组合前面——后者说明链路是真的通了，前者可能只是虚假同步多。

粘滞锁定。 全量扫描要试几十个组合，每个都要跑一遍解调，代价不小。但参数一旦调准，短时间内不会变。所以命中之后锁住获胜的 (Δ_f, f_c) 组合，后续只做一次快扫验证；连续`iq_decoder_autotune_lock_max_miss`（默认 3）次未命中才回退全量扫描：

算法 2 粘滞锁定

```
if 已有锁定组合 then
    用锁定组合试解
    if 解出期望命令 then
        未命中计数 ← 0；return 结果
    else
        未命中计数 ← 未命中计数 + 1
        if 未命中计数 < 阈值 then
            return 空结果（本周期跳过）
        end if
        清除锁定；未命中计数 ← 0
    end if
end if
全量扫描候选网格，按式 (8) 评分
if 最优候选解出期望命令 then
    锁定该组合
end if
return 最优结果
```

比赛配置里信息波的自动调谐实际是关闭的（`iq_autotune_mode: off`）——因为连续流路径已经在跑，快照解码器默认不启用。自动调谐主要服务两个场合：干扰波那一路（`quick` 模式），以及赛前用离线扫描确定参数基准值。

2.3.8 干扰波的时间分集

干扰波有一个可以利用的特性：它承载的 `0x0A06` 是**恒定的**，以 10 Hz 重复发送同一份密码帧，只在密钥更换时变一次。这意味着同一份帧会有大量噪声副本。

我们实现了两级分集，都遵守同一条底线：**分集只产生候选，CRC 仍是唯一裁判。**

第一级：软符号等增益合并。 这一级发生在**比特判决之前**。定时恢复输出的是浮点电平，硬判决把它压成 0/1 就丢掉了置信度信息。做法是：

1. 在同一组 (SPS, offset) 符号网格上定位所有检测到的副本——这样各副本互为整数比特偏移，采样点能一一对齐；
2. 对反相的副本把软符号取负（极性校正）；
3. 直接平均。

平均软符号（类似对数似然比）比逐比特多数表决更有效：置信度高的符号会连续地压倒噪声符号，而不是”一比特一票”。这就是软判决相对硬判决的约 2 dB 合并增益。

第二级：逐比特多数表决。 软合并之后如果还是过不了 CRC，再退一步做硬表决。这一级的关键是不能被无关载荷污染——干扰波里除了密码帧还有大量随机填充，如果把所有 15 字节载荷放在一起表决，结果就是噪声。

做法是先取出现次数最多的那份副本作为种子，只把与种子 Hamming 距离 ≤ 20 bit（120 bit 里的 20）的副本纳入表决簇：

```
seed_hex = Counter(r.hex() for r in raws).most_common(1)[0][0]
seed = np.unpackbits(np.frombuffer(bytes.fromhex(seed_hex), dtype=np.uint8))
hamming = np.count_nonzero(bit_rows != seed, axis=1)
cluster = bit_rows[hamming <= int(cluster_hamming_bits)]
if cluster.shape[0] < int(min_copies):
    return []
voted = (cluster.mean(axis=0) >= 0.5).astype(np.uint8)
```

随机填充载荷与密码帧的 Hamming 距离期望是 60 bit，远在 20 之外，所以聚簇能有效地把它们排除。要求至少 3 份副本才表决，少于这个数表决没有统计意义。

两级都只增不减。 表决出的共识载荷仍然要送进`RefereeFrameAssembler`走完整的 CRC8 + CRC16，通过了才作为一帧产出，并标注`source='time_diversity_vote'`以便追溯。没通过就丢弃。任何情况下都不会因为”表决结果看起来合理”而放行——一个错误的密钥提交上去是要浪费一次验证机会的。

> **这套机制只对干扰波成立。**它的前提是”池子里有一份主导的、内容恒定的帧”。信息波的五条命令是轮转发送、内容每帧都在变的，拿去做跨帧表决只会得到垃圾。所以`vote_repeated_frames`只在干扰波那一路打开。这类”看起来通用、其实依赖特定前提”的优化，一定要把前提写在代码注释里，否则很容易被后来的人误用到另一条链路上。

2.4 系统架构与通信

2.4.1 为什么用 ROS 2 包 GNU Radio

GNU Radio 的 flowgraph 本身是一个自成一体的运行时，完全可以写成一个独立进程直接开串口。我们没这么做，而是用 ROS 2 节点把每个 flowgraph 包起来，理由有三条：

1. **视觉和无线电必须通信。**视觉侧是 Python + TensorRT 的进程，无线电侧是 GNU Radio 的进程，两者要交换目标位置。自己写 socket 协议要处理序列化、重连、话题发现，ROS 2 现成的。

2. **一个进程只能有一个串口持有者**。视觉和无线电都想给裁判系统发数据，但串口是独占设备。用 ROS 2 之后，串口的持有权明确归 `rm_referee_serial_node`，其他节点通过话题发请求（第 2.4.4 节）。
3. **诊断和录制需要旁路**。每个节点把状态发到`~/status`，Dashboard 和录制器都只是订阅者，加一个观测者不需要改被观测的节点。比赛录制订阅了 16 个话题（第 2.8.2 节），如果这些数据是进程内的，就得在每处埋点。

代价是多了一层 ROS 的依赖和构建复杂度。有一个具体的坑记在这里：

Humble 的 colcon 与新版 setuptools 冲突。 ROS 2 Humble 的 `colcon-python-setup-py` 依赖系统 setuptools 提供的`develop --editable` 扩展，而用户 site-packages 里较新的 setuptools 会静默顶掉它，导致`colcon build --symlink-install` 失败，报错信息还指向别处。我们在`setup.py` 里加了一段 re-exec 逻辑：只在 colcon 的 editable 构建时用`PYTHONNOUSERSITE=1` 重新进入一次，绕开用户 site-packages。这段代码不影响正常打包、测试和运行，但删掉会让`--symlink-install` 报错。如果在别的机器上遇到类似的构建失败，先检查`pip list | grep setuptools` 的版本。

2.4.2 节点划分

表 11: ROS 2 节点一览

入口点	作用
rm_gfsk_node	RX1 信息波接收与解调，发布帧与状态
rm_gfsk_interference_node	RX2 干扰波接收与解调
rm_referee_serial_node	裁判串口收发，0x0305 / 0x0301 组帧，唯一串口持有者
rm_radar_integration	视觉与无线电双源融合、双倍易伤决策
rm_auto_password_node	六位干扰密码生成与下发
rm_match_recorder	按裁判赛况自动录制 IQ 与话题事件
rm_match_dashboard	Web 运维界面（HTTP + SSE）
rm_ganraoyuan_node	干扰源发射 flowgraph（仅实验室）
rm_iq_replay_tx_node	把录好的 IQ 回放到 TX
rm_offline_iq_scan	离线 IQ 参数扫描与解码
rm_virtual_link_test	不接硬件的全链路自环测试

话题命名统一用相对命名空间：~/status、~/frames、~/referee_bridge、~/setters_json、~/spectrum、~/filtered_spectrum；串口节点额外有~/rx_frames、~/tx_frames、~/raw_rx、~/game_status。每个 flowgraph 节点提供~/start、~/stop、~/reload、~/status_trigger 四个Trigger 服务。

有一层划分值得单独说：core/ 目录下全是**不依赖 ROS 的纯逻辑**——协议解析、CRC、帧同步、自动调谐、融合、双倍易伤决策、录制实现。nodes/ 只做 ROS 的胶水（声明参数、订阅发布、定时器）。这样划分的直接好处是测试：19 个 pytest 文件里绝大多数不需要 ROS 环境也不需要硬件（第 4.2 节）。

2.4.3 视觉与无线电的通信契约

这是本文唯一详细展开视觉侧的地方。

比赛默认referee.transport: radio_ros：**视觉不开串口**，把目标遥测以 20 Hz 发到同机 ROS 2 话题/rm_radar_algorithm/telemetry，由无线电侧融合

后限速到 4.8 Hz 写裁判串口。

遥测消息是 JSON，schema 标识为`shark.radar.telemetry.v1`：

```
{
  "schema": "shark.radar.telemetry.v1",
  "source_time": 1754716252.331,
  "side": "red",
  "vision": {
    "camera_ready": true, "model_ready": true,
    "fps": 11.8, "processing_fps": 11.8, "camera_fps": 12.1,
    "inference_ms": 62.4, "filter_type": "kalman"
  },
  "robots": {
    "B1": {"position_cm": {"x": 1820, "y": 640}, "valid": true,
           "state": "measured", "confidence": 1.0,
           "observed_at": 1754716252.331},
    "B2": {"position_cm": {"x": 0, "y": 0}, "valid": false,
           "state": "missing", "confidence": null,
           "observed_at": null}
  }
}
```

十二台机器人（红蓝各 6 台，编号 1/2/3/4/6/7）每台一项，永远全部出现——没观测到的报`state: "missing"` 而不是省略字段。这样接收侧不需要处理”字段缺失”和”明确无观测”的区别。

最关键的设计是`state` 这个字段有四种取值，而不是一个布尔：

表 12: 视觉状态的四种取值

state	含义
measured	当前帧真实检测到，坐标可信
occlusion_hold	被遮挡，用的是遮挡前缓存的位置
blind_prediction	在盲区，用的是运动模型外推的位置
missing	没有可用位置

为什么必须区分这四种：因为**这四种状态的可信度完全不同**，而视觉在敌方目标上承担的正是”无线电不可用时的回退”这个角色（第 2.5 节）——回退成不成立，取决于这一帧的视觉观测是真的看见了，还是只是缓存或外推。一个`measured` 是真实观测，接得住；一个`blind_prediction` 只是运动模型外推，接过来也未必对。如果遥测只报”有效/无效”，融合层就没有办法做这个区分。

更实际的一点是：occlusion_hold 和blind_prediction 在双倍易伤决策里不被信任（第 2.6.1 节）——一台车”可能还在那个区域”不足以支撑消耗一次宝贵的双倍易伤机会。

反向通道有两条：

- /rm_referee_serial_node/referee_bridge ——无线电侧把解析好的裁判数据（比赛状态、血量、标记进度等）转发给视觉，视觉用它显示界面信息、判断阵营。
- /rm_radar_algorithm/radar_cmd ——视觉可以请求触发双倍易伤，带一个request_id；无线电侧执行后通过/rm_referee_serial_node/tx_frames 回 ACK，视觉据此知道请求是成功写入了还是被拒。这条通道目前实际不用（决策已统一收到无线电侧），但接口保留。

legacy_serial 是回退方案，视觉直接开/dev/ttyUSB0 自己发。只在无线电侧完全不可用时使用。

2.4.4 串口独占与协议限速

裁判串口有两条硬约束：设备独占，以及 0x0305 最高 5 Hz。

独占。 OptionalSerialPort 在打开设备前先拿一个文件锁。串口路径用 udev 规则固定成/dev/shark-referee，避免/dev/ttyUSB* 的序号随插拔顺序漂移。另外启动器本身也有互斥保护：用flock 加锁并检查 PID 文件，发现旧的 wrapper 还在跑就先把它停掉，防止两份链路同时抢 SDR。

限速要做两层。 融合节点的发送定时器设在 4.8 Hz，已经低于 5 Hz 上限。但这不够——ROS 定时器在执行器阻塞后会**补偿性地连续触发**，把攒下的回调一次性跑完。这种情况下瞬时速率可以远超 5 Hz。

所以在定时器之外还加了一层基于单调时钟的闸门：

```
# ROS timers may catch up with back-to-back callbacks after executor stalls.
# Keep a second, monotonic protocol guard so no burst can exceed official 5 Hz.
if self._last_send_monotonic and \
   monotonic_now - self._last_send_monotonic < 1.0 / 5.0:
    return
```

串口节点侧还有第三层同样的检查，并统计被拦下的次数（blocked_0305_rate_count）暴露到状态里。三层看似冗余，但超发是会被判罚的，而定时器补偿这种行为在压力测试下才出现，不能指望它不发生。

原始帧发送被限制。 ~/send_frame 话题拒绝直接发送 0x0301/0x0121 组合——雷达指令必须走~/send_radar_cmd，这样才会经过状态机校验（比如”密码更

换是否被允许”、”双倍易伤剩余机会是否为零”）。绕过校验直接构帧是能编译通过的，所以这道限制写在了运行时。

2.4.5 两个应用的边界

仓库里有两个独立应用，配置、端口和安全策略都不共享：

表 13: 比赛接收应用与实验室发射应用的边界

	apps/match_rx	apps/lab_tx
用途	比赛接收	实验室发射调试
端口	:8765	:8780
裁判串口	打开	**永不打开**
射频发射	无	有，需安全确认

比赛链路是纯接收的，代码层面不具备任何发射能力。发射只存在于 LabTX，且需要在面板上逐次勾选射频安全确认——配置文件里的 `lab_antenna_confirm` 始终提交为`false`，保证克隆仓库后一启动不会发射。另有`mute_tx.sh` 做赛后硬静音（TX 衰减拉满并关闭 TX 本振）。

LabTX 的存在解决了一个很实际的问题：**没有官方发射源时如何调接收链路**。它按规则手册的波形参数发射合法的信息波与干扰波，包括第 2.1.4 节说的字节流切片行为。本文引用的实验室实测数据都是在这套发射台上得到的。

衰减默认值是台架校准过的：信息波 61 dB，干扰波 11 dB，对应规则手册的 −60 / −10 dBm。这里有一个版本迁移的坑：GNU Radio 3.10.7 的 gr-iio 用满量程 I/Q，61 dB 对应旧版 3.10.1 在 37 dB 时的台架电平（约 +24 dB 的迁移差）。升级 GNU Radio 之后如果发现接收端信号突然变强或变弱，先查这个。

2.5 双源融合

融合的目标很明确：**每台机器人给出一个最终坐标，并且任一源失效时仍能继续标记**。实现在`core/radar_integration.py`，是一个不依赖 ROS 的纯状态类。

两条链路的分工不是对等的：

- **无线电是敌方坐标的主来源**——全场无遮挡，直接给场地坐标，但只有对方六台车，且速率低（各命令 10 Hz）。
- **视觉是无线电不可用时的回退，同时是己方坐标的唯一来源**——受遮挡限制，但它能看到己方目标，而无线电只播对方信息。

所以敌方目标上视觉的角色是”接得住”，而不是”更准”；己方目标上视觉没有替代品，必须独立保持可用。

2.5.1 时效模型

融合不做加权平均，而是**按时效性选择来源**。一个源在某一时刻要么可信要么不可信，中间态没有意义——无线电断流后的坐标不会因为”和视觉平均一下”变得更接近真实位置，视觉被遮挡时缓存的位置也不会。把两个不同时刻、不同可信度的观测混在一起，得到的是一个既不准也不新的坐标，而且出问题时无法判断是哪个源坏了。选一个更可信的，同时把选择结果暴露给操作者，反而更实用。

三个时效阈值：

表 14: 融合的时效阈值

参数	默认	含义
`vision_timeout_sec`	0.5 s	视觉观测超过此时长视为失效
`radio_timeout_sec`	1.0 s	无线电坐标超过此时长不再参与发送
`radio_ghost_sec`	2.0 s	过期后仍在地图上画影子的时长

这三个数字的来源：视觉以 20 Hz 发遥测，0.5 s 相当于连续丢 10 帧，到这个程度基本可以断定视觉侧出了问题而不是偶发抖动。无线电各命令 10 Hz，1.0 s 相当于连续丢 10 帧，同样的逻辑，但阈值放宽一倍是因为无线电本来就更容易丢包（第 4 节的实测丢包率 10%–25%）。

`radio_ghost_sec` 是纯粹为了操作者判断的——过期的无线电点位不再参与发送，但仍然在地图上画出来（用区别于正常点位的样式），这样操作者能看出”链路是刚断的还是彻底没了”。这个区别在现场很重要：刚断可能是天线被人挡了一下，等一等会恢复；彻底没了要去查硬件。

2.5.2 来源选择规则

对每台机器人：

算法 3 单个目标的来源选择

visual_fresh ← (视觉数据年龄 ≤ 0.5s) ∧ 视觉坐标有效
radio_fresh ← (无线电数据年龄 ≤ 1.0s) ∧ 无线电坐标有效
if 该目标不是本方机器人且 radio_fresh **then**
    来源 ← 无线电
else if visual_fresh **then**
    来源 ← 视觉
else
    来源 ← 无（坐标置零）
end if

两处需要解释：

敌方优先无线电，本方只能用视觉。 因为无线电只播对方的信息。红方基座的发射源播蓝方数据，所以红方雷达从无线电只能拿到蓝方位置。本方机器人的坐标（0x0305 的后 12 个 uint16）只能来自视觉。代码里用`is_ally`判断这一点，本方目标直接跳过无线电分支。

敌方为什么以无线电为主、视觉为回退。 无线电给的是官方发射源播出的坐标，全场无遮挡、不依赖标定、不受光照影响，覆盖上没有死角。视觉受遮挡限制——RoboMaster 场地有高地、围挡和各种掩体，一台车躲进掩体视觉就丢了，而无线电完全不受影响。考虑到标记判定是”坐标落在目标附近”而不是”精确到厘米”，无线电的覆盖优势直接决定了它是主来源。

视觉在敌方目标上的价值是**兜住无线电断流的那几秒**：射频链路会因为天线被挡、增益不合适、邻道干扰变强而短时退化（第 2.2 节），这时候有一个哪怕精度一般的位置，也比完全没有点位要好——标记进度是按时间累积的，断一秒就少一秒。

2.5.3 阵营热同步与一个必须做的清理

阵营（红/蓝）决定了两件事：RX 的中心频点，以及 0x0305 载荷里”前 6 个是敌方、后 6 个是本方”的映射方向。

阵营可以自动从裁判系统同步（`auto_sync_side: true`）——0x0201 里的`robot_id`落在 1–11 是红方，101–111 是蓝方。也可以在 Dashboard 顶栏手动切，运行中生效。

这里有一个必须处理的问题：**阵营切换时已缓存的无线电数据必须清掉。**

```
if changed:
    # Radio packets are assigned opponent names using own_side. Never
    # retain a packet that was decoded under the previous orientation.
    self.radio = {}
    self.radio_received_at = None
```

原因是无线电解出的 0x0A01 只有”对方六台车”这个语义，具体映射到 R1–R7 还是 B1–B7 取决于我方阵营。如果阵营从红切到蓝而不清缓存，那些原本标记为 B 的坐标会被当成 R 继续用——结果是把敌方位置报成了本方位置。这个 bug 如果发生在比赛中，现象是地图上点位整体镜像，而且很难第一时间看出来是阵营问题。

视觉遥测不需要这个清理，因为它自己带side 字段，每帧都重新声明。

2.5.4 0x0305 载荷的构造

0x0305 是 24 个 uint16（小端），依次为敌方六台车的 (x, y)、本方六台车的 (x, y)。构造时有两条规则：

- **坐标合法性检查**：$0 \le x \le 2800$，$0 \le y \le 1500$（单位 cm，对应 2800×1500 的场地），且不能全零。不合法的一律填 $(0, 0)$，表示”无观测”。
- **全零则不发**：如果 24 个值全是零，说明一个目标都没有，这时候不发帧——发一个全零的 0x0305 没有意义，只占用速率配额。

还有一处安全检查：如果配置的阵营与裁判系统检测到的阵营不一致，直接阻止发送并报configured_side_mismatch。宁可不发，也不能按错误的阵营发——后者等于把本方位置报给了对手那一侧的槽位。

2.6 比赛决策

融合给出坐标之后，无线电侧还要做三件事：申请双倍易伤、向本方机器人广播哪些目标无敌、生成并提交干扰密码。这三件都是”什么时候做”比”怎么做”更难的问题。

2.6.1 双倍易伤自动决策

规则：雷达每累计使对方机器人易伤 1 分钟，获得 1 次触发双倍易伤的机会；触发后当前所有来自雷达的易伤数值翻倍，持续 30 秒；每局最多触发 2 次。

所以这是一个”最多两次、每次 30 秒、不用就浪费”的资源。决策目标是让这 60 秒落在对方最吃亏的时候。

我们的实现（core/double_vulnerability.py）是一个纯状态机，三个触发条件：

表 15: 双倍易伤的三个触发条件

条件	判据
敌方进入 ROI 停留	任一敌方英雄 / 步兵 / 哨兵在批准的 ROI 掩码内停留 ≥ 3 秒
本方英雄骤降血量	滚动 3 秒窗口内本方英雄累计掉血 ≥ 100 HP
终局兜底	比赛剩余时间 ≤ 30 秒时，把剩余机会全部消耗或排队

ROI 停留：为什么要 3 秒。 ROI 掩码是一张不规则的二值图（打包在`assets/`下），标出”敌方停在这里就值得翻倍”的区域——主要是我方前哨站和基地附近的进攻位。用不规则掩码而不是矩形，是因为场地本身不规则，矩形要么覆盖不全要么带进无关区域。

要求停留 3 秒而不是瞬时进入，是为了避开”路过”。一台车穿过区域和一台车驻守在区域里，战术意义完全不同，但瞬时位置判断分不出来。3 秒对应大约 30 帧无线电更新，足够区分。

掩码是**面向本方**的：图像永远以我方一侧在下方。官方场地坐标系是红方朝向的，所以红方需要做 180° 旋转，蓝方直接映射：

```
# The approved mask is always oriented with our side at the bottom.
# Official field coordinates are red-oriented, so red requires a 180-degree
# turn while blue maps directly into the side-oriented image.
if side == 'red':
    x = FIELD_X_CM - x
    y = FIELD_Y_CM - y
```

这样一张掩码同时服务两个阵营，不需要维护两份、也不会出现”改了红方忘了改蓝方”。

位置来源必须可信。 这是把第 2.4.3 节那四种视觉状态用起来的地方。只有下面两种情况的位置被信任：

- 来源是无线电，且无线电数据新鲜（`radio.fresh`）；
- 来源是视觉，且视觉新鲜且 `state == 'measured'`。

`occlusion_hold` 和`blind_prediction` 都不算——”这台车可能还在区域里”不足以消耗一次机会。

还要排除已阵亡和已无敌的目标。 这用的是 `0x0A05` 最后 5 字节的主要状态（第 2.1 节）。只有状态为 0（存活）才算合格目标，并且这份状态本身必须新鲜（默认 2.5 秒内）。如果状态过期或缺失，一律**不合格**——宁可错过一次触发，也不要对一台已经无敌的车翻倍。

英雄掉血：为什么用滚动窗口的最大值。 判据是"3 秒窗口内累计掉血 ≥ 100 HP"。实现上不是比较首尾两个采样，而是取窗口内的最大血量作为基线：

$$\text{drop} = \max_{t \in W} \text{HP}(t) - \text{HP}(t_{\text{now}}) \tag{9}$$

用最大值而不是首值，是因为英雄可能在窗口内先回血再掉血（补给区回血）。用首值会把回血量抵消掉一部分，低估实际的掉血幅度。

终局兜底：不用就浪费。 双倍易伤持续 30 秒，比赛最后 30 秒时如果还有剩余机会，留着一定是浪费。所以到这个时间点就把剩余机会全部消耗掉，或者排队（官方 0x0121 协议允许在生效期间再排一次）。这一条是纯粹的"止损"逻辑，不追求最优时机。

一个必须用计数确认的细节。 请求发出去之后要确认它被受理了，否则可能重复请求浪费机会。直觉的做法是看 0x020E 第 2 位（是否生效）的上升沿。但这样有一个漏洞：那个上升沿也可能来自**并发的或手动的**请求——比如操作手在客户端手动触发了一次。如果用生效边沿确认，我们的自动请求会误以为自己成功了，而实际上它可能被拒（因为已经有一次在生效中）。

所以我们用 0x020E 第 0–1 位的**剩余机会计数下降**作为权威确认：

```
# The remaining-opportunity counter is the authoritative acknowledgement.
# An active edge alone could belong to a concurrent/manual request and
# must not unlock a second automatic increment.
if available < available_before:
    self._pending = None
```

计数下降说明确实消耗了一次机会。在计数下降之前，状态机停在 request_pending，不会发第二次。

裁判数据过期时的处理。 比赛状态流断了怎么办？我们的处理是分两档：

- **清掉进行中的观测**——3 秒停留计时和血量窗口都作废。因为这些判断依赖连续的数据，断流期间的时间不能算。
- **但不清掉已发出的请求**——否则数据一恢复就会重复发一次。

这两者的区别在代码里是_clear_live_conditions() 和 _reset_for_new_match() 两个方法，前者只清观测，后者连请求一起清，只在确认比赛重新开始时调用。

2.6.2 无敌目标广播

这是我们自定义的一条团队消息，走裁判 0x0301，data_cmd_id = 0x0234。

用途：把 0x0A05 解出的敌方无敌状态告诉本方机器人。操作手看不到对方是不是处于无敌（比如刚复活的无敌保护期），打一台无敌的车是纯浪费弹丸和暴露位置。

载荷设计得极简——**1 个字节，5 个比特**：

表 16: 0x0234 无敌目标载荷

比特	对应目标
0	对方英雄
1	对方工程
2	对方 3 号步兵
3	对方 4 号步兵
4	对方哨兵
5–7	未使用

主要状态为 2 或 3（两种无敌）时置 1，0 或 1（存活 / 阵亡）时置 0。

发送方式是轮询：6 个接收者（本方 1/3/4/5/6/7 号机器人）各 3 Hz，所以帧率是 $3 \times 6 = 18$ fps。用轮询而不是广播，是因为 0x0301 的`receiver_id` 是单播字段。

数据过期时发全零（全部标记为非无敌）而不是不发。理由是：不发的话操作手界面上会保持上一次的状态，可能显示一台其实已经可以打的车还是无敌——这比没有信息更糟。全零至少让操作手回到”按常规判断”。

2.6.3 干扰密码生成

流程：订阅 0x020E，在”允许更换密钥”位（第 5 位）的上升沿生成一个新的六位密码，经`send_radar_cmd` 下发，两次之间至少间隔 5 秒。

密码用`secrets.choice` 生成（而不是`random`），从一个刻意裁剪过的字母表里取：

```
DEFAULT_PASSWORD_ALPHABET = ''.join(
    ch for ch in string.ascii_letters + string.digits if ch not in '0O1Il'
)
```

剔掉`0`、`O`、`1`、`I`、`l` 五个字符的理由很实际：这些字符在赛场大屏和操作手界面的字体下难以区分。密码是要被人读、被人核对的，误读一位就白发一次。剩下 57 个字符，六位组合空间 $57^6 \approx 3.4 \times 10^{10}$，远超一场比赛的猜测能力，牺牲这 5 个字符不影响安全性。

这一条是”面向人”而不是”面向机器”的设计。纯从密码学角度看，剔字符是降低了熵。但实际的失效模式不是被暴力破解，而是人读错一位。类似的判断在这个项目里还有几处——比如诊断面板按阶段拆开计数，也是因为真正的瓶颈是人的判断速度而不是信息量。

2.7 现场诊断：比赛接收台 Dashboard

这是本项目投入最多、也最实用的一块。

无线电雷达最难的不是写解调，而是**在比赛现场几十秒内判断出链路到底哪一环出了问题**——天线没对准、增益不合适、解调参数漂了、还是串口没通。这四种情况的处理方式完全不同，而如果只有一个”收到 N 帧”的数字，根本区分不出来。Dashboard 就是为这件事做的。

2.7.1 几个设计取舍

浏览器而不是 Qt。 前端是纯静态页 + SSE 推送，GNU Radio 的原生 Qt sink 只在`--native-gui`诊断时启用。浏览器渲染频谱和瀑布图比 Qt sink 流畅得多（频谱 25 Hz 推送、1024 点 FFT），而且不占 flowgraph 的主线程；检测不到图形桌面时还能退化成 headless Web 模式继续服务。

分层而不是堆参数。 两页分工明确：比赛时盯「雷达地图」看结果，出问题才切「无线电监控」看过程。所有配置和进程管理收进高级抽屉，不占用比赛视野。

可运行中改。 阵营、干扰等级、RX 增益都在页面上热切换，不用重启链路——比赛暂停的时间窗口很短，重启一次链路（等 SDR 初始化、等解调收敛）的代价太大。

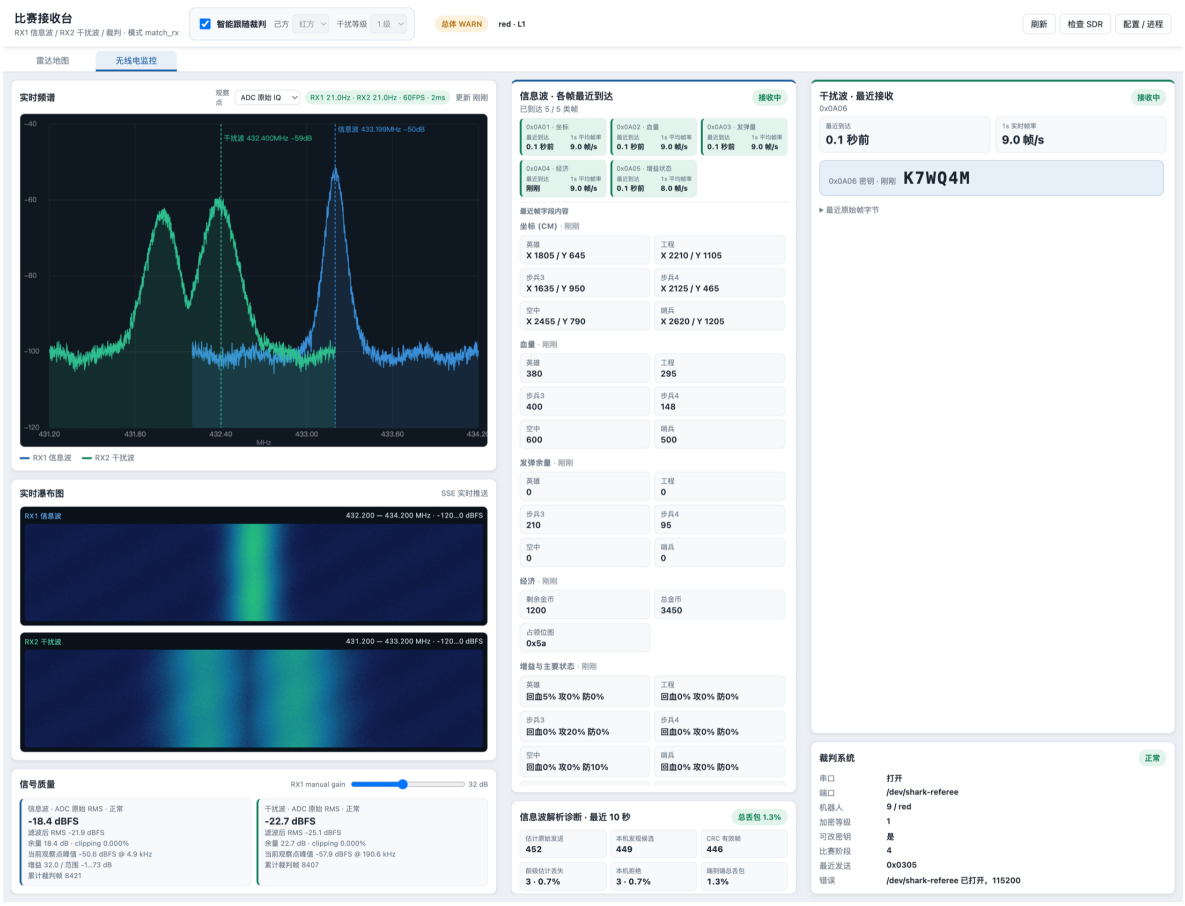


图 3: 「无线电监控」页。上方是 RX1 信息波与 RX2 干扰波叠加在同一频率轴上的实时频谱，自动标注两路峰值频点与电平；中部是两路各一张瀑布图；下方左侧是信息波各条命令的最近到达时间与实时帧率，右侧是按阶段拆开的解析诊断。界面效果示意，图中频谱与瀑布图是按 433.200 / 432.200 MHz 频率规划构造的合成信号，不是实测底噪与载波。

2.7.2 按阶段拆开的解析诊断

这是整个面板最有价值的一块。数据源就是第 2.3.5 节组帧器产生的那些事件，在 10 秒滑动窗口内分阶段计数：

表 17: 解析诊断的阶段划分与对应的排查方向

阶段	这一项高说明什么
帧同步失败	有信号但比特错太多。增益过高削顶、极化失配、SPS 漂了
长度域失败	接入码匹配上了但长度域不对。通常是虚假同步，或反相判断有误
CRC16 失败	帧结构对了但数据有错。信噪比不够，微调增益或天线指向
命令长度失败	CRC 过了但长度与 cmd_id 不匹配。随机填充偶然构成合法帧，正常现象
有效	成功解出

关键在于" 全是零" 和" 某一项很高" 是完全不同的信息:

- 各项全零 → 根本没收到信号。查天线、频点、SDR 是否在取流。
- 帧同步失败高、其余为零 → 收到了但解不出。查增益、极化、解调参数。
- CRC16 失败高、有效帧少 → 链路通了但信噪比不够。微调天线指向。
- 有效帧正常但地图没点 → 不是无线电的问题，往融合或串口方向查。

如果只报一个总帧数，这四种情况看起来都是" 帧数少"。

2.7.3 按现象定位: 现场用法

把上面的逻辑整理成一张表，贴在比赛机旁边:

表 18: 按现象定位故障

现象	看哪里	常见原因
地图上完全没有无线电点位	「各帧最近到达」全是—	天线没对准 / 增益太低 / 频点不对
频谱有信号但解不出帧	解析诊断的「帧同步失败」	增益过高削顶、极化失配、SPS 漂了
帧同步过了但没有有效帧	解析诊断的「CRC16 失败」	信噪比不够，微调增益或天线指向
解析成功率忽高忽低	信号质量面板 + 增益滑条	AGC 被误开，或天线在波束边缘
只有部分命令收到	「各帧最近到达」逐条比对	正常现象，各命令发送周期不同；长期缺失才是问题
地图有点但没发出去	「裁判串口」徽章	串口未打开 / 设备路径漂了
无线电点位变成过期影子	坐标对照表的「年龄」列	链路刚断，年龄超过 1 秒就不再参与发送

2.7.4 架天线的正确用法

这一条单独写出来，因为它是 Dashboard 最出乎意料的用途，也是让八木天线真正发挥增益的关键一步。

做法：**一个人转天线，另一个人盯着「解析诊断」的有效帧计数和「信号质量」的成功率**，找到峰值再锁死。

为什么必须这么做：八木的波束窄，目视对准的误差足以损失可观的增益，而" 看起来对着" 和" 增益最大" 往往差几度。用有效帧计数做反馈，把一个几何对准问题变成了一个可观测的极值搜索问题。

这个方法在实验室阶段就验证过——同一个位置，凭感觉对准和用计数找峰值，有效帧率能差出可观的幅度。三级干扰下这个差距更明显，因为链路余量本来就小。

2.7.5 HTTP 接口

面板本身是静态页 + 几个 JSON 接口，也可以直接`curl`：

表 19: Dashboard HTTP 接口

方法	路径	作用
GET	/api/state	全量状态快照
GET	/api/spectrum	单次频谱数据
GET	/api/spectrum-stream	频谱 SSE 流
GET	/api/radar-stream	地图与融合结果 SSE 流
GET	/api/performance	推送性能统计
GET	/api/check_sdr	触发 SDR 可达性检测
POST	/api/config	保存配置到config.yaml
POST	/api/start / /api/stop	启停链路
POST	/api/clear	清空运行时状态

配置有三层，优先级从低到高：config.yaml（持久层，被 Git 跟踪）、环境变量（一次性运行时覆盖，永不写回）、Dashboard 保存（写回config.yaml）。仓库刻意不支持隐式的local.env ——那种隐藏文件曾让不同机器的部署静默产生分歧，排查起来非常费时。需要批量覆盖时用显式的MATCH_RX_ENV_FILE 指向一个文件。

2.8 比赛录制与离线复现

2.8.1 为什么要录 IQ

射频问题的现象往往不可重现：同一个位置、同一套参数，今天能解出来明天就不行。靠现场记忆复盘几乎不可能。

所以我们把比赛全过程的原始 IQ 录下来。有了 IQ，赛后可以把任意一段重新灌进解调链路，试不同的参数组合，看当时那一帧到底是为什么解不出来。本文里几个参数的最终取值（增益 32 dB、低通 260 kHz、SPS 候选范围）都是这么定的。

2.8.2 录制触发

自动跟随裁判系统赛况，不需要人操作：

- 连续 2 帧 0x0001 的game_progress == 4（比赛中）→ 开始
- 连续 2 帧game_progress == 5（比赛结束）+ 10 秒后摇 → 停止
- 硬上限 900 秒

要求连续 2 帧而不是 1 帧，是为了避免单帧误码触发。10 秒后摇是为了录到结算画面——有时候问题恰好出在最后几秒。

录制内容三类：原始 IQ、ROS 话题事件流（16 个话题）、配置快照。配置快照这一项很容易被忽略但很重要——没有它，赛后看到一段解不出的 IQ，无法确定当时用的是哪套参数。

2.8.3 存储策略

表 20: 录制的存储策略

类型	策略
清单、配置快照、事件日志、完成标记	始终写内盘，保证重启后仍在
原始 IQ	外接盘可读写且剩余 $\geq$ 15 GB 时用外接盘；否则回落内盘

分开处理的理由：元数据很小但必须在，IQ 很大但可以降级。如果统一写外接盘，盘没插上就什么都没有了；统一写内盘，一场比赛就能占掉几个 GB。

2.8.4 IQ 落盘格式

直接存 complex64 太大：2 MS/s × 8 字节 × 2 路 × 600 秒 = 19 GB。我们做了三级压缩：

1. **整数抽取回 1 MS/s** ——数据量减半。必须是整数抽取，因为离线重放要精确还原相位关系；重采样会引入插值误差，让”复现”变成”近似”。
2. **块缩放 CI8** ——每个数据块独立计算一个 float 缩放因子，然后把 I/Q 各量化到 int8。数据量再减到 1/4。用块级而不是全局缩放，是因为滤波后的信号幅度动态范围很大，全局缩放会让安静段落几乎全部量化成零。
3. **zlib level 1** ——轻量压缩，主要压掉量化后的冗余。用 level 1 而不是更高，是因为这是在接收线程的旁路上跑的，不能占太多 CPU。

最终双路 10 分钟约 2.4 GB。每个数据块带 CRC32，读的时候能发现损坏而不是静默给出错误数据。

IQ 取自各接收机**已有的**滤波流，不会再开第二个进程去抢 SDR ——这一点很重要，两个进程同时`iio_readdev` 同一台设备会互相干扰。队列满时**丢录制块而不是阻塞接收**，录制永远不能影响比赛链路。

2.8.5 三条离线复现路径

表 21: 三条离线复现路径

命令	用途
`rm_offline_iq_scan`	扫描一段 IQ，在参数网格上找出能解出命令的组合。用来定参数基准值
`start_iq_replay_tx.sh`	把录好的 IQ 回放到 TX，让接收端重新走一遍完整链路（含射频）
`rm_virtual_link_test`	完全不接硬件，跑通编码 → 解码全链路

三条路径的覆盖范围递减但便利性递增: IQ 回放包含真实的射频环节（能复现天线和增益问题），离线扫描只覆盖解调之后（但可以快速试很多参数），虚拟链路只验协议逻辑（但完全不需要硬件，可以进 CI）。

3 硬件要求

3.1 本队配置

表 22: 无线电雷达的硬件配置

用途	设备	天线 / 说明
信息波接收 (RX1)	ANTSDR E310	六单元八木天线，垂直极化，正对官方发射源。`ip:192.168.9.110`
干扰波接收 (RX2)	ANTSDR E310	玻璃钢全向天线。`ip:192.168.9.111`
信息波发射（实验室）	ADALM-Pluto	实验室短天线。`ip:192.168.2.1`
干扰波发射（实验室）	ADALM-Pluto	实验室短天线。`ip:192.168.3.1`
裁判系统	USB 转串口	udev 固定为`/dev/shark-referee`，115200
数据盘	外接 2 TB	挂载于`/media/shark/Resources`，存 IQ

比赛机只需要两台 RX。两台 TX 仅在实验室复现发射链路时使用。表中的 IP 与设备路径是本队部署值，换机器需按自己的硬件改`apps/*/config.yaml`与 udev 规则。

3.2 最低要求与替代方案

表 23: 硬件的最低要求

项目	要求
SDR	任意 AD9361 板子（Pluto 及兼容板均可）。**必须带网口或独立供电**，理由见第 2.2 节
信息波天线	定向天线，垂直极化。六单元八木是我们的选择，更多单元更好但架设难度上升
干扰波天线	全向即可，对信噪比要求低
主机	无线电侧不需要 GPU。解调是纯 CPU 的，一台普通笔记本足够
操作系统	Ubuntu 22.04 LTS

关于主机性能：无线电这一侧的计算量不大。2 MS/s 的连续解调在现代 CPU 上占用很低，真正吃资源的是视觉侧的 TensorRT 推理。如果只做无线电雷达，不需要独立显卡。

3.3 软件环境

表 24: 软件环境版本

项目	版本
ROS	ROS 2 Humble
GNU Radio	3.10.7
gr-iio	随 GNU Radio 3.10.7
libiio / libad9361	系统包
Python	3.10（系统 Python）
浏览器	Chrome / Chromium（频谱与瀑布图主显示）

GNU Radio 版本会影响发射电平的标定。3.10.7 的 gr-iio 用满量程 I/Q，同一个衰减设置在 3.10.1 上对应的实际功率不同（我们台架实测约 24 dB 的差距）。如果按本文的衰减值配置发射台却发现电平不对，先确认 GNU Radio 版本。接收侧不受此影响。

3.4 部署要点

串口固定命名。 避免/dev/ttyUSB* 的序号随插拔顺序漂移:

```
# 先查串口芯片的 idVendor / idProduct
udevadm info -a -n /dev/ttyUSB0 | grep -E 'idVendor|idProduct' | head -4

sudo tee /etc/udev/rules.d/99-shark-referee.rules <<'EOF'
SUBSYSTEM=="tty", ATTRS{idVendor}=="1a86", ATTRS{idProduct}=="7523", SYMLINK+="shark-referee
    "
EOF
sudo udevadm control --reload-rules && sudo udevadm trigger
```

SDR 网络。 设备走 USB 网络或以太网，给主机对应网卡配同网段静态地址，然后验证可达:

```
iio_info -u ip:192.168.9.110 | head -5
```

通电自检。 apps/match_rx/doctor.sh 输出ok / warn / bad 三级结果，检查项包括: 配置是否被 Git 跟踪、配置是否与当前 commit 一致、ros2 / python3 / zstd 命令、GNU Radio 运行时、两台 RX 的 iio 可达性（5 秒超时）、裁判串口是否存在、录制目录与剩余空间。

”配置是否与当前 commit 一致”这一项容易被当成多余，但它抓过一类问题: 某台机器上改了配置没提交，换机器部署时那份改动没跟过去，现场才发现参数不对。有未提交改动时报warn，提醒确认这是有意的部署差异。

4 性能评估与测试

4.1 实测结果

丢包率的计算方法见第 2.3.6 节，即用裁判帧头的 8 位序号估计解调前丢失的帧数，再加上被 CRC 拒掉的帧。

4.1.1 实验室: 按规则手册标称功率复现发射

用 LabTX（第 2.4.5 节）按规则手册的参数发射，信息波 −60 dBm、干扰波 −10 dBm，两路同时发射:

表 25: 实验室实测（严格按规则手册功率发射）

条件	结果
八木天线架设良好、三级干扰波	可稳定解析信息波，丢包率约 **25%**

这个条件是偏严苛的：三级干扰是最难的等级（干扰波距信息波仅 400 kHz，见第 2.2.3 节），而实验室复现的发射功率严格卡在手册标称值上。所以 25% 更像是链路余量的**下界参考**，而不是赛场表现的预期值。

在这个丢包率下系统仍然可用，原因是各命令都以 10 Hz 重复发送——`0x0A01` 位置信息丢掉 25% 之后仍有约 7.5 Hz 的有效更新率，而融合层的无线电超时阈值是 1.0 秒（第 2.5 节），远比这个间隔宽松。

4.1.2 赛场实际环境

表 26: 赛场实测

条件	结果
官方发射源，实际赛场环境	可稳定以 **10 fps** 解析信息波（即各命令的标称发送速率），丢包率 **10%** 以下

赛场表现明显好于实验室复现，主要原因是**官方发射源的信息波实际功率高于手册标称值**。这一点对备赛有直接意义：如果在实验室按标称功率能勉强解出来，到赛场上通常会有额外余量；反过来，如果实验室都解不出来，赛场也不用指望。

”稳定 10 fps” 意味着解析速率已经达到发送速率上限——链路不再是瓶颈，剩下的丢包对融合层没有实质影响。

4.2 测试策略

`src/rm_radio_ros/test/` 下有 19 个 pytest 文件。测试策略的核心是**把需要硬件的和不需要硬件的严格分开**。

4.2.1 不需要硬件的部分（pytest）

得益于`core/` 与`nodes/` 的划分（第 2.4 节），下面这些都是纯软件可测的：

表 27: 纯软件测试覆盖

测试文件	覆盖内容
test_rm_protocol.py	协议编解码、CRC8/CRC16、帧同步、组帧器的各条失败路径
test_continuous_gfsk_demod.py	解调链路的参数换算（含第 2.3.2 节那个抽取灵敏度）
test_rx_autotune.py	候选网格生成、评分函数、粘滞锁定
test_offline_iq_scan.py	离线扫描与时间分集表决
test_radar_integration.py	融合的来源选择、时效判定、阵营切换时的缓存清理
test_double_vulnerability.py	双倍易伤状态机的全部分支
test_invincible_targets.py	0x0234 载荷构造与状态映射
test_referee_serial_node.py	串口节点的限速、原始帧拦截、状态机校验
test_match_recording.py	录制格式、块缩放、CRC32、回读
test_virtual_link.py	编码 → 解码全链路自环
test_launch_and_script_safety.py	启动脚本的安全性（发射确认位、串口策略）

运行方式:

```
source install/setup.bash
python3 -m pytest src/rm_radio_ros/test -v
```

其中test_launch_and_script_safety.py 是一类比较特别的测试——它检查的不是功能而是安全约束：比赛应用的启动脚本里不能出现发射相关的调用、LabTX 的配置里lab_antenna_confirm 必须提交为false、比赛链路不能引入 TX flowgraph。这类约束靠 code review 容易漏，写成测试之后每次提交都会检查。

4.2.2 需要硬件的部分（实测脚本）

放在apps/lab_tx/run_*.sh，不进 pytest：

表 28: 需要真实硬件的实测脚本

脚本	用途
`check_four_sdr.sh`	检测四台 SDR 是否全部可达
`run_four_sdr_live_test.sh`	四台 SDR 联调实测
`run_demod_ab_test.sh`	两种解调路径 A/B 对比
`run_rx1_gain_sweep.sh`	RX1 增益扫描，用来定第 2.2.5 节那个 32 dB
`run_dual_tx_raw_rx1_capture.sh`	双发射下抓 RX1 原始 IQ
`run_generated_link_test.sh`	生成载荷跑链路测试

实测脚本刻意忽略环境变量覆盖。 `run_four_sdr_live_test.sh` 与 `run_demod_ab_test.sh` 会无视外部设置的环境变量，强制使用提交在配置文件里的那份增益与采集参数。理由是：如果实测能被环境变量影响，两次测试的结果就不可比——而这类脚本的输出（比如增益扫描的最优点）是要写进配置的，必须保证它反映的是仓库里那份配置的真实表现。

4.2.3 虚拟链路自环

`rm_virtual_link_test` 是覆盖面最广的一个测试：它把编码端和解码端在内存里对接，走完整的”构裁判帧 → 按 15 字节切片 → 加接入码与长度域 → 提取空中载荷 → 组帧 → 校验 → 解析字段”流程，完全不接硬件。

它能抓到的最典型问题是第 2.1.4 节那个跨包边界——如果组帧器假设了”一个包一条命令”，虚拟链路立刻就会失败，因为编码端刻意模拟了不整除的切片行为。

5 总结与展望

5.1 项目总结

无线电雷达在 2026 赛季是全新方向，我们在赛季初期基本处于”不知道能不能收到”的状态。能走到现在这一步，很大程度上是靠前面几支队伍愿意把方案和踩坑公开出来 [3, 4, 5]，我们在这个基础上继续摸索，补上了几处自己遇到、也确实卡了一阵的问题。整理下来大概是四方面。

把接收链路的每一级参数都追到了来源。 从 gr-iio 取样到六条空中命令解析全部走通，过程中尽量避免留下”不知道为什么是这个值”的常数。链路里几处不显然的换算——抽取之后调频灵敏度要乘回 2、为什么用带通而不是低通、两级 AGC 为什么都设得很慢——都在本文对应小节写了推导或台架依据。这些不算什么新方法，但当初每一处都花掉了不少时间，写清楚或许能让后来者少走一遍。`core/` 与`nodes/` 的划分让协议层完全脱离 ROS 和硬件，带来的实际好处是大部分测试可以在任何机器上直接跑。

对官方频率规划的一点应对经验。 规则把强 50 dB 的干扰波放在距信息波 400 kHz 处，这是本赛季射频侧最主要的矛盾。我们的做法是空间选择性、幅度稳定性、频率选择性三层叠加，在三级干扰下能稳定解析。方法本身都是常规手段，谈不上创新，但”**信息波必须关掉 AGC**”这一条我们确实是绕了远路才定位到的——它的现象（成功率忽高忽低、天线不动也变）非常容易把人往天线方向误导。如果本文只有一条经验值得看，大概是这一条。

两项利用场景特性的接收增强。 自动调谐把”手工对频偏和 SPS”变成在参数网格上自动搜索，配合粘滞锁定兼顾冷启动的鲁棒性与稳态延迟；干扰波那一路利用它恒定重发的特性做时间分集，用软符号合并加聚簇多数表决，在单帧过不了 CRC 时仍能恢复密钥。软合并本身是通信里的常规做法，我们做的只是把它接到这个具体场景上，并把”CRC 始终是唯一裁判”这条底线守住。

一些围绕现场可用性的工程投入。 把解析过程按阶段拆开计数，让”没收到信号”和”收到但解不出”能被直接区分开——这个想法很简单，但对现场排查的帮助超出我们预期。配套的 Web 面板里，最实用的反而不是频谱和瀑布图，而是”一个人转天线、一个人盯有效帧计数”这个用法，它把天线对准从凭感觉的几何问题变成了可观测的极值搜索。比赛全程录制 IQ 让赛后能复现任意一帧的解调过程，本文里几个参数的最终取值都是靠离线重放定下来的。

需要说明的是，本文的方案肯定不是最优解。下面这些不足我们自己清楚，只是本赛季没有时间或数据去解决。

5.2 已知不足

融合是”选一个”而不是真正的融合。 现在的做法是按时效性在主来源与回退之间切换，没有做状态估计层面的融合。更好的做法是把两个源都作为观测量喂进一个卡尔曼滤波器，按各自的噪声协方差加权——尤其是在无线电与视觉同时可用的时段，两路观测互相约束能压掉一部分抖动，而现在这段时间里视觉的信息实际上是被丢掉的。没做的原因是两个源的时间戳对齐比较麻烦（视觉的观测时刻、无线电的空口时刻和收到时刻都不同），本赛季优先保证了可用性。

解调侧没有做载波频偏的闭环跟踪。 自动调谐是开环搜索：搜到一组好参数就锁住，靠”连续未命中”触发重搜。如果频偏在比赛过程中缓慢漂移（温漂是真实存在的），会经历一段”锁定的参数已经不是最优但还没到触发重搜”的窗口。更规范的做法是加一级 FLL 做连续跟踪。

丢包率估计有固有偏差。 第 2.3.6 节说明过：8 位序号在连续丢超过 127 帧时会失效，而且会话切分依赖一个经验阈值。这个估计用来判断链路好坏够用，但不适合当作精确指标去和别队的数字横向比较。

三级干扰下的余量仍然偏紧。 实验室 25% 的丢包率虽然可用，但离舒适还有距离。可能的改进方向包括更高增益的天线、加一级模拟前端滤波（比如 SAW 滤波器），以及在解调侧引入判决反馈均衡。这几条我们都还没验证过。

双倍易伤的触发条件是启发式的。 现在的三个条件（ROI 停留、英雄掉血、终局兜底）是根据我们对战术的理解定的，没有基于历史比赛数据做过优化。理论上”什么时候触发收益最大”是可以量化的问题——需要统计各时段的实际易伤累积速率，本赛季没有足够的数据支撑。

只有单场地的数据，参数不一定通用。 本文的实测数据来自我们自己的实验室和有限的赛场机会。不同场地的电磁环境差异可能很大（尤其是其他队伍的图传和电源），这套参数在别的场地不一定合适。建议使用者用 `run_rx1_gain_sweep.sh` 在自己的场地重新扫一遍增益，而不是直接照抄本文的 32 dB。

5.3 未来展望

状态估计层面的双源融合。 如上所述，把视觉与无线电作为两路观测喂进统一的滤波器，按噪声特性加权。这需要先解决时间戳对齐——可以试着利用裁判系统 0x0001 里的`sync_timestamp` 字段做共同时基。

载波与定时的闭环跟踪。 在现有的开环自动调谐之外加一级 FLL/PLL，让频偏跟踪从”定期重搜”变成”连续跟踪”。GNU Radio 有现成的块，主要工作量在于和现有的粘滞锁定机制协调，避免两套机制互相打架。

解调侧的软信息利用。 现在信息波走的是硬判决路径。虽然没有 FEC 可以利用软信息做译码，但软信息至少可以用于质量指示——比如统计判决余量的分布来实时估计信噪比，比现在用丢包率间接推断更直接，也能更早预警链路劣化。

把诊断经验沉淀成自动建议。 现在那张”按现象定位”的表是给人看的。既然各阶段的计数都已经在系统里，理论上可以让系统自己给出建议（”帧同步失

败占比 80%，建议检查增益与极化”）。这对新队员上手会有帮助，也能减少对”有经验的人恰好在场” 的依赖。

6 鸣谢

无线电雷达是这个赛季从零摸索的方向，研发过程中得到了很多队外的帮助。

感谢**华东理工大学**、**应急管理大学**、**南京邮电大学**、**北师港浸大**等学校的同学，以及 **RM 雷达群**的各位群友。在频点确认、天线架设、解调参数和各种踩坑经验上的交流，帮我们省掉了大量试错时间——尤其是在赛季初期，连” 信息波到底能不能收到” 都还是个问号的阶段，这些交流提供了很关键的信心。

感谢愿意把无线电部分的探索公开出来的几支队伍：**华南师范大学 PIONEER 战队** [3] 的整体方案开源，**北师港浸大 Navigator 战队** [4, 5] 的射频前端分享与开发记录。本赛季无线电是全新方向，公开资料不多，这几份分享的价值很难被高估。

视觉一侧感谢**厦门理工学院 PFA 战队** [6] 与**香港科技大学 ENTERPRIZE 战队** [7] 的开源工作。我们的视觉链路在检测、投影与轨迹预测上大量参考了这两份成果，本文虽然不展开视觉部分，但这份帮助必须写明。

也感谢开源社区：GNU Radio 与 gr-iio 让自建射频链路成为一件普通队伍做得起的事，ROS 2 省掉了我们自己造进程间通信的功夫。

最后感谢 SHARK 战队的全体成员。雷达这个兵种在队里的可见度不高——它不上场、没有炫技的镜头，出问题时才被注意到。感谢队里在这件不那么显眼的事上给予的支持与耐心。

7 后记

无线电雷达在 2026 赛季是一个全新方向，本文写的方案肯定不是最优解，但我们尽量把**每个决定背后的理由**都写清楚了——包括那些走错了又走回来的部分。对后来者来说，知道” 为什么不那样做” 往往比知道” 要这样做” 更有用。

如果这套东西对你有帮助，欢迎交流；如果你发现了本文里的错误，更欢迎指出来。

- **开源仓库**

  - 聚合仓库：https://github.com/JNU-SHARK/shark-radar-system
  - 无线电子系统：https://github.com/JNU-SHARK/shark-radar-radio
  - 视觉子系统：https://github.com/JNU-SHARK/shark-radar-vision
- 作者　郭兰鑫，江南大学霞客湾校区 SHARK 战队

两个子系统均采用 MIT License，视觉部分基于 YOLOv5 二次开发。欢迎自由使用、修改与再分发；如果在设计中受益于本项目，希望能在你的开源引用中提及来源——这对后续的开源生态有帮助。

射频使用声明　本项目的发射功能仅用于 RoboMaster 竞赛规则允许的场景与实验室环境。使用前请确认符合当地无线电管理规定。LabTX 的多重安全确认设计（会话级确认开关、配置永不提交确认位、`mute_tx.sh` 硬静音）请勿绕过。

参考文献

[1] DJI 大疆创新. *RoboMaster 2026 机甲大师超级对抗赛比赛规则手册 V2.0.1*. 2026-06-29. 波形参数见第 5.6 节表 5-23 与附录 1「雷达信息波特性」.

[2] DJI 大疆创新. *RoboMaster 2026 机甲大师高校系列赛通信协议 V2.0.0*. 2026-06-26. 雷达无线链路命令见第 1.6 节.

[3] 华南师范大学 PIONEER 战队. 【*RM2026 雷达无线电开源*】. https://bbs.robomaster.com/article/1882741.

[4] 北京师范大学-香港浸会大学联合国际学院 Navigator 战队. 【*BNBU Navigator 2026 雷达射频二阶段开源*】+ 开发计划. https://bbs.robomaster.com/article/1884147.

[5] 北京师范大学-香港浸会大学联合国际学院 Navigator 战队. *Navigator RMUC 2026 雷达射频开发小记*. https://bbs.robomaster.com/article/1885744.

[6] 厦门理工学院 PFA 战队. *RM2026 厦理 PFA 雷达站算法开源*. https://bbs.robomaster.com/article/1884254.

[7] 郭子霖, 周思宇（香港科技大学 ENTERPRIZE 战队）. 香港科技大学 *ENTERPRIZE 战队 RoboMaster 2025 赛季雷达站算法开源报告*. 2025-08-04. https://bbs.robomaster.com/article/761138.

[8] GNU Radio Project. *GNU Radio 3.10 Documentation*. https://www.gnuradio.org/doc/doxygen/.

[9] K. Mueller, M. Muller. *Timing Recovery in Digital Synchronous Data Receivers*. IEEE Transactions on Communications, vol. 24, no. 5, pp. 516–531, 1976. DOI: 10.1109/TCOM.1976.1093326.

[10] Analog Devices. *AD9361 RF Agile Transceiver Reference Manual (UG-570)*. https://www.analog.com/media/en/technical-documentation/user-guides/ad9361.pdf.

[11] Open Robotics. *ROS 2 Humble Hawksbill Documentation*. https://docs.ros.org/en/humble/.